

Rebirth: Full Cold-Start, Concurrency, Portfolio-Reduction, and UI Performance Guide

Copy-ready implementation guide for cold start, connector concurrency, Portfolio reduction, data aggregation, Stock redesign, and deployment

Prepared 18 August 2026

Contents

Implementation order	10
1. The key diagnosis	10
2. Understand the four different meanings of “worker”	11
3. Create a safe branch and remove the accidentally committed guide	11
4. Add a standalone cold-start watcher	12
File: tools/s04_watch_startup.py	12
4.1 Diagnose a freeze that happens before /healthz exists	14
File: tools/s05_profile_app_import.py	14
5. Make startup begin server-side after the cold page mounts	15
File: ui/s09_factory.py	15
Location: immediately after	15
Replace cube_page_body() with	15
Do not make a partial edit to pnl_page_body() here	15
6. Build the shared hero in cold-loading mode immediately	16
File: ui/s09_factory.py	16
Function: serve_layout()	16
7. Preserve the hero until the financial page consumes revision 1	16
File: ui/s07_events.py	17
Replace the complete hydrate_shared_refresh_shell callback decorator and function with	17
8. Leave a completed hero visible long enough to paint	18
File: assets/s02_app.js	18
Function: finishRefreshProgress()	18
Function: recoverReadyBootstrap()	18
9. Add a targeted dashboard read so startup does not copy every large frame	19
File: core/s02_pipeline.py	19
Location: immediately after the existing FrameRead dataclass	19
Inside RiskRefreshManager , replace the existing control_snapshot property with the following block and add read_dashboard() immediately below it	19

In core/s02_pipeline.py::__all__ add	19
File: ui/s01_contracts.py	20
Add after FrameReadProtocol	20
In RefreshManagerProtocol add	20
In __all__ add	20
File: ui/s09_factory.py	20
File: ui/s07_events.py	21
File: ui/s04_components.py	21
9.1 Reuse the factory's prepared dashboard cache inside Risk callbacks	22
File: ui/s07_events.py	22
Replace _RiskDataCache.__init__() with	22
Replace _RiskDataCache.replace_frame() completely	22
Add a parameter to register_callbacks()	22
Replace the _RiskDataCache construction	23
File: ui/s09_factory.py	23
Add the loader to the existing register_callbacks() call	23
10. Avoid the defensive full-snapshot copy that cold startup discards	23
File: core/s02_pipeline.py	23
In the RiskRefreshManager.refresh() signature add	23
File: ui/s01_contracts.py	24
File: ui/s07_events.py	24
Function: StartupCoordinator._run()	24
11. Remove duplicate initial Risk and P&L; table construction	25
File: ui/s04_components.py	25
Function: build_layout()	25
File: ui/s09_factory.py	26
12. Remove the intentional one-second Risk product delay	26
File: s01_app.py	26
13. Add bounded connector task workers inside the one process	27
File: core/s02_pipeline.py	27
Add these imports near the top	27

Add these parameters to RiskRefreshManager.__init__()	27
Add this validation inside __init__() before assigning instance fields	27
Add these assignments after the existing loader assignments	28
Add this method inside RiskRefreshManager , immediately before _load_product_risk()	29
14. Parallelize independent Risk product source calls	30
File: core/s02_pipeline.py	30
Replace _load_product_risk() with	30
In RiskRefreshManager.refresh() , find the block beginning with	30
15. Parallelize real or fake per-Underlying market calls	31
File: core/s02_pipeline.py	32
Replace _load_product_market_open() with	32
Replace _load_product_market_status() with	33
16. Pass connector worker settings from the feed composition root	34
File: feeds/s01_sources.py	34
Add these helpers immediately above build_production_refresh_manager()	34
Inside build_production_refresh_manager() add	34
Add these arguments to the RiskRefreshManager(...) call	34
17. Replace the recovered single-worker async bridge	35
File: adapters/s01_common.py	35
Add this block before class RiskSource	36
Add run_async to __all__	37
18. Parallelize the three independent real Portfolio sources	37
File: feeds/s01_sources.py	37
Add these imports	37
Extend the core.s01_schema import with	37
Add the following complete implementation immediately above get_portfolio_config()	38
Replace the executable body of get_portfolio_config() with	41
Configure real-client I/O timeouts	41
19. Clear the Portfolio cache only for an explicit Portfolio/force refresh	42
File: core/s02_pipeline.py	42
Add this module helper near _load_portfolio_config()	42

In refresh_portfolios() , immediately before _load_portfolio_config(...) , add	42
In refresh() , within the final block, replace	42
20. Do not add generic “async CSV reads”	42
21. Collapse Portfolio before publishing dashboard_frame	43
File: core/s02_pipeline.py	44
Replace the complete to_dashboard_frame() function with	44
22. Remove Portfolio from the governed UI selector registry	46
File: ui/s02_constants.py	46
Replace the import	46
Replace the complete block from PORTFOLIO_UI_FIELD = ... through FILTER_DIMENSION_FIELDS = ... with	46
23. Remove Portfolio from Quick Search	46
File: core/s03_search.py	47
Remove PORTFOLIO_COLUMN from the core.s01_schema import	47
Replace PIVOT_INDEX_COLUMNS , GOVERNANCE_COLUMNS , and RISK_ONLY_INDEX_COLUMNS with	47
Replace QUICK_RISK_FILTER_COLUMNS with	47
In _risk_pivot_catalog_frame() , replace required with	47
In build_search_catalog() , delete	47
File: ui/s04_components.py	47
24. Remove the Portfolio-specific unmapped-books callback dependency	48
File: ui/s07_events.py	48
25. Make P&L; history callbacks independent of the number of filters	48
File: ui/s08_plevents.py	49
Replace the render_historical_pl_hierarchy() signature and first lines with	49
Replace the render_historical_pl_chart() signature and first lines with	49
File: ui/s06_plview.py	49
File: ui/s14_pl_filters.py	49
26. Migrate saved views from five filters to four	50
File: assets/s01_style.css	50
27. Allow partially populated Credit measures	50
File: core/s02_pipeline.py	50
Function: get_product_risk()	50

File: ui/s03_aggregate.py	51
Function: credit_measure_available()	51
Function: credit_measure_values()	51
28. Move the natural tenor sorter into the core schema	51
File: core/s01_schema.py	51
File: ui/s03_aggregate.py	52
File: core/s02_pipeline.py	52
29. Normalize conflicting or missing tenor orders instead of rejecting them	52
File: core/s02_pipeline.py	53
Replace _validate_market_tenor_orders() with	53
File: ui/s03_aggregate.py	54
Replace _resolved_tenor_orders() with	54
30. Average duplicate market quotes before merging Open and Current	55
File: core/s02_pipeline.py	55
Add immediately after _normalize_market_tenor_orders()	55
Replace get_product_market_open() with	56
Replace get_product_market_status() with	57
Replace _merge_validated_market_legs() with	58
31. Display Open, Current, and Move at Reported Underlying	59
File: ui/s03_aggregate.py	59
Function: should_show_sum()	59
32. Replace the custom Stock hierarchy with a P&L-style; Aggregate Stock page	59
32.1 Aggregate duplicate raw Stock rows instead of rejecting them	60
File: core/s07_stock.py	60
Add immediately before compare_stock_snapshots()	60
Replace the first four lines inside compare_stock_snapshots()	60
32.2 Remove Portfolio after the Stock mapping join	60
File: core/s07_stock.py	60
Replace MAPPED_STOCK_COMPARISON_COLUMNS	60
Replace STOCK_FILTER_COLUMN_BY_KEY	61
Add immediately before map_stock_comparison_portfolios()	62

Replace map_stock_comparison_portfolios() completely	63
32.3 Load current Stock, prior Stock, and Portfolio mapping concurrently	63
File: ui/s10_stock.py	63
Add imports near the top	63
Add above load_stock_page_data()	64
Replace load_stock_page_data() completely	65
32.4 Add the P&L-style; Aggregate Stock table	66
File: ui/s10_stock.py	66
Replace the .s02_constants import with	66
Add below STOCK_FILTER_NOTE	66
Add immediately above build_stock_page_from_data()	67
32.5 Replace the loaded Stock page body	70
File: ui/s10_stock.py	70
Replace build_stock_page_from_data() completely	70
32.6 Replace the Stock placeholder	71
Replace build_stock_page_placeholder() completely	71
32.7 Replace the complete Stock shell	72
Replace build_stock_page_shell() completely	72
32.8 Replace the Stock filtering callback	74
File: ui/s09_factory.py	74
Replace the Stock imports from .s10_stock	74
Replace the complete current filter_stock_table() callback decorator and function	75
32.9 Reduce the process-local Stock cache	76
File: ui/s09_factory.py	76
32.10 Update build_stock_table() for the Portfolio-free mapped frame	76
File: ui/s10_stock.py	76
32.11 Delete the obsolete Stock hierarchy code	76
32.12 Replace the pure Stock page helpers and final exports	77
File: ui/s10_stock.py	77
Replace build_stock_page() completely	77
Replace build_stock_page_from_sources() completely	78
Replace the complete ui/s10_stock.py::__all__ list with	78

File: core/s07_stock.py	79
Replace the complete core/s07_stock.py::__all__ list with	79
33. Keep the fake connector simple and deterministic	79
33.1 Keep fixture worker counts at one	79
33.2 Optional: reduce repeated stat() calls on network storage	79
File: feeds/s01_sources.py	79
34. Keep one Gunicorn process and make request threads configurable	81
File: s04_server.py	81
35. Add faster Dash JSON serialization	82
File: requirements.txt	82
36. Add final-stage timing logs	82
File: core/s02_pipeline.py	82
In RiskRefreshManager.refresh() , find the _progress_step() call whose function name is _commit_full_snapshot and whose stage is final	82
File: ui/s07_events.py	83
37. Reduce the startup watchdog from forty minutes	83
File: ui/s09_factory.py	83
38. Optional cleanup of recovered AI/comment blocks	83
39. Update the tests	84
39.1 Portfolio registry and release	84
File: tests/s19_risk_filters.py	84
39.2 Add a dashboard collapse integration test	84
39.3 Saved views	85
File: tests/s23_saved_views.py	85
39.4 Partial Credit	85
File: tests/s06_ui.py	85
39.5 Market duplicate and tenor tests	85
File: tests/s04_market.py	85
39.6 Reported Underlying market aggregate	86
File: tests/s06_ui.py	86
39.7 Cold startup shell	86

File: tests/s12_startup.py	86
39.8 Targeted dashboard read	87
39.9 Startup no-copy return	87
39.10 Connector concurrency	87
39.11 Stock	88
File: tests/s17_stock.py	88
39.12 Fake fixture mode	88
40. Run checks in this order	88
40.1 Compile	88
40.2 Focused tests	88
40.3 Complete suite	89
40.4 Stage the deployment bundle	89
41. Recommended environment settings	89
41.1 Fake/local fixture mode	89
41.2 Initial real-source mode	89
42. Production acceptance checklist	90
Cold start	90
Cardinality	90
Credit and market	90
Concurrency	90
Stock	90
Server	91
43. Expected result	91
44. Primary references	91

Repository audited: streamlitdash/Rebirth

Code baseline: commit e715d7c5eefd562e8eea8066ea1a3128e382e5ed

Current main at the time of this audit: bd8ae49b5c743471afcc831f7e6bb50f56e35eb3

The only code difference between those two commits is the added file

`rebirth_cohesive_implementation_guide.md` ; the Python application code is unchanged.

This guide combines the previous implementation guide with a deeper cold-start and connector-concurrency audit. It is written as an implementation sequence. Follow the steps in order.

Implementation order

Apply the guide in this sequence:

- 1 Add the cold-start watcher and source-level timeouts.
- 2 Make startup server-owned and repair the hero/revision handoff.
- 3 Stop copying full snapshots and constructing initial tables twice.
- 4 Add bounded connector task workers, keeping fake mode at one worker initially.
- 5 Parallelize the three real Portfolio sources and configure native API timeouts.
- 6 Collapse Portfolio and remove every analytical UI dependency on it.
- 7 Apply the Credit, tenor and duplicate-market changes.
- 8 Replace the Stock hierarchy with Aggregate Stock.
- 9 Tune Unicorn threads and serialization only after the functional changes pass.
- 10 Run the focused tests, complete suite, cold-start watcher and production acceptance checklist.

1. The key diagnosis

Portfolio is **not proven to be the direct cause of Cube never opening**.

Portfolio currently causes a very large **final analytical frame**, a large Quick Search catalogue, large Dash callback payloads, and expensive hierarchy construction. It can therefore make the last part of startup and page navigation appear frozen.

However, the initial refresh obtains market data by looping over **unique Underlyings**, not Portfolios. If startup is stuck while the active function is a market connector, deleting the Portfolio filter will not fix that connector call.

The current cold-start path has several independent risks:

- 1 `StartupCoordinator.schedule_start()` exists, but the production page builders do not call it. Startup therefore depends on browser JavaScript or a Dash interval reaching `/startz` or the startup callback.
- 2 Open and Current market calls are made sequentially for every Underlying within every product.
- 3 The recovered real async bridge uses a single-worker executor. If enabled as written, it serializes all async calls and can deadlock if it is ever called recursively from its own worker.
- 4 Real synchronous connectors have no guaranteed hard I/O timeout in the shared framework. The 2,400-second startup watchdog reports a stall but cannot terminate a blocked Python call.
- 5 The final refresh stage loads portfolio configuration, attaches reporting mappings, builds the analytical frame, builds the search catalogue, commits the snapshot, and then defensively copies every large DataFrame even though the startup coordinator discards the returned copy.
- 6 The startup page and shared-shell callbacks can both copy/materialize large snapshot data at the same time.

- 7 `build_layout()` constructs the initial Risk hierarchy and Aggregate P&L table, then mounted callbacks build them again.
- 8 The shared hero starts hidden, and successful completion is hidden after only 300 milliseconds.
- 9 `dashboard_frame` still contains raw Portfolio, and Quick Search still indexes it.

The correct solution is therefore **both**:

- make cold-start ownership, timeouts, progress, and task concurrency reliable; and
- reduce the amount of data that reaches the final dashboard and browser.

2. Understand the four different meanings of “worker”

Do not mix these concepts:

Worker type	What it does	Recommended setting
Gunicorn worker	A separate Python process serving HTTP requests	Keep 1 for now
Gunicorn thread	A request-handling thread inside the one process	Keep 4 , then benchmark 8
Connector task worker	A <code>ThreadPoolExecutor</code> task that overlaps blocking source I/O	Fake 1/1 ; real start Risk 2 , Market 4
Async event-loop worker	One background event-loop thread that can run many awaiting coroutines	One loop thread, many coroutines

Do **not** set `Gunicorn workers=2` yet. Each Gunicorn worker would own a separate refresh manager, snapshot, revision, search catalogue, caches, locks, Stock state, and startup coordinator. Requests could then land on different financial revisions.

The multi-worker improvement in this guide is **connector task concurrency inside the one authoritative process**.

3. Create a safe branch and remove the accidentally committed guide

The repository test `tests/s13_publish.py::test_root_readme_is_the_only_markdown_document` requires `README.md` to be the only Markdown file in the repository. The current `main` contains `rebirth_cohesive_implementation_guide.md` , so that test will fail.

Run:

```
git checkout -b fix/cube-cold-start-and-performance
git rm rebirth_cohesive_implementation_guide.md
git status
```

Keep this downloaded guide outside the repository.

Create a baseline test log:

```
python -m pytest -q > baseline-pytest.txt
python -m ruff check . > baseline-ruff.txt
```

Commit the clean baseline before changing application code:

```
git add -A
git commit -m "Remove temporary implementation guide from runtime repository"
```

4. Add a standalone cold-start watcher

This distinguishes “no refresh started”, “connector is stuck”, “final DataFrame work is slow”, and “revision committed but browser handoff is stuck”.

File: `tools/s04_watch_startup.py`

Create the file with the following complete content:

```
"""Start and follow one Cube cold-start transaction through its HTTP endpoints."""

from __future__ import annotations

import argparse
import json
import sys
import time
from urllib.error import HTTPError, URLError
from urllib.parse import urljoin
from urllib.request import Request, urlopen

def _url(base_url: str, prefix: str, endpoint: str) -> str:
    base = base_url.rstrip("/") + "/"
    normalized_prefix = prefix.strip("/")
    relative = f"{normalized_prefix}/{endpoint}" if normalized_prefix else endpoint
    return urljoin(base, relative)

def _json_request(url: str, *, method: str = "GET") -> dict[str, object]:
    request = Request(
        url,
        method=method,
        headers={"Accept": "application/json"},
    )
    with urlopen(request, timeout=35) as response:
        content_type = response.headers.get("Content-Type", "")
        if "application/json" not in content_type.casefold():
            raise RuntimeError(
                f"{url} returned {content_type or 'unknown content type'}"
            )
        payload = json.loads(response.read().decode("utf-8"))
    if not isinstance(payload, dict):
        raise RuntimeError(f"{url} did not return a JSON object")
    return payload

def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--base-url", required=True)
    parser.add_argument(
        "--prefix",
        default="/",
        help="Public Dash request prefix, for example /proxy/8050/",
    )
    parser.add_argument("--poll-seconds", type=float, default=1.0)
    parser.add_argument("--timeout-seconds", type=float, default=900.0)
    args = parser.parse_args()

    start_url = _url(args.base_url, args.prefix, "startz")
    progress_url = _url(args.base_url, args.prefix, "progressz")
    health_url = _url(args.base_url, args.prefix, "healthz")

    try:
        started = _json_request(start_url, method="POST")
    except (HTTPError, URLError, TimeoutError, RuntimeError) as error:
        print(f"Could not request startup: {error}", file=sys.stderr)
        return 2

    print(
        "Start request:",
    )
```

```

    json.dumps(started, indent=2, sort_keys=True, default=str),
)

deadline = time.monotonic() + max(1.0, args.timeout_seconds)
previous = None
while time.monotonic() < deadline:
    try:
        progress = _json_request(progress_url)
    except (HTTPError, URLError, TimeoutError, RuntimeError) as error:
        print(f"Progress endpoint error: {error}")
        time.sleep(max(0.1, args.poll_seconds))
        continue

    fingerprint = (
        progress.get("revision"),
        progress.get("startup_phase"),
        progress.get("running"),
        progress.get("stage"),
        progress.get("function_name"),
        progress.get("source_type"),
        progress.get("underlying"),
        progress.get("product_index"),
        progress.get("product_total"),
        progress.get("current"),
        progress.get("total"),
        progress.get("error"),
    )
    if fingerprint != previous:
        previous = fingerprint
        print(
            " | ".join(
                [
                    f"revision={progress.get('revision')}",
                    f"startup={progress.get('startup_phase')}",
                    f"running={progress.get('running')}",
                    f"stage={progress.get('stage')}",
                    f"function={progress.get('function_name')}",
                    f"source={progress.get('source_type')}",
                    f"underlying={progress.get('underlying')}",
                    f"unit={progress.get('product_index')}/{progress.get('product_total')}",
                    f"work={progress.get('current')}/{progress.get('total')}",
                    f"error={progress.get('error')}",
                ]
            )
        )

    revision = int(progress.get("revision") or 0)
    phase = str(progress.get("startup_phase") or "")
    if revision > 0 and phase == "succeeded":
        print("Cold start committed successfully.")
        return 0
    if phase == "failed":
        print("Cold start failed.", file=sys.stderr)
        return 1

    time.sleep(max(0.1, args.poll_seconds))

try:
    health = _json_request(health_url)
except Exception as error: # diagnostic tool: preserve the original timeout
    health = {"health_error": str(error)}
print(
    "Timed out while watching startup. Final health:",
    json.dumps(health, indent=2, sort_keys=True, default=str),
    file=sys.stderr,
)
return 3

if __name__ == "__main__":
    raise SystemExit(main())

```

Run locally with:

```

python tools/s04_watch_startup.py \
    --base-url http://127.0.0.1:8050 \
    --prefix /

```

For a proxied deployment, use the public Dash request prefix, not the internal Flask route prefix.

Interpret the output as follows:

Observation	Meaning
startup=idle , revision=0	No startup signal reached the coordinator
stage=market_open or market_status , same Underlying for a long time	A market source call is blocked or serially slow
stage=final , revision=0	Mapping, DataFrame collapse, search catalogue, snapshot copy, or final validation is slow
revision=1 , browser still on cold shell	Browser/Dash handoff or full layout materialization is slow
startup=stalled	The watchdog fired; the original connector still owns the writer
startup=failed	The transaction raised and no revision was published

4.1 Diagnose a freeze that happens before /healthz exists

If /healthz never responds, the Portfolio-expanded DataFrame is not yet the cause: the refresh pipeline has not started. The worker is blocked while importing the application or constructing the WSGI object.

File: tools/s05_profile_app_import.py

Create the file with the following complete content:

```

"""Import the Cube WSGI application and dump stacks if import construction hangs."""

from __future__ import annotations

import faulthandler
import importlib
import sys
import time
import traceback

def main() -> int:
    faulthandler.enable(all_threads=True)
    faulthandler.dump_traceback_later(
        60.0,
        repeat=True,
        file=sys.stderr,
    )
    started = time.perf_counter()
    try:
        module = importlib.import_module("s01_app")
        server = getattr(module, "server", None)
        if server is None:
            raise RuntimeError(
                "s01_app imported but did not expose server"
            )
    except Exception:
        traceback.print_exc()
        return 1
    finally:
        faulthandler.cancel_dump_traceback_later()

    print(
        "s01_app imported successfully in "
        f"{time.perf_counter() - started:.3f}s"
    )
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Run:

```

python tools/s05_profile_app_import.py \
2> cold-import-stacks.log

```

Also collect Python import timings:

```
python -X importtime -c "import s01_app" \
2> cold-import-times.log
```

Interpretation:

- A stack in `colossus.connect`, `MRX`, `ConfigManager`, an HTTP client, or another private SDK means source I/O was accidentally placed at module scope.
- A stack repeatedly importing large optional libraries means the complete recovered import inventory was enabled unnecessarily.
- A successful fast import followed by a stuck `/progressz` stage means the problem is in the refresh pipeline instead of WSGI construction.

The real connector implementation must follow this rule:

```
Importing s01_app may construct connector callables and clients that perform no
network I/O. The first network call must occur only inside RiskRefreshManager.refresh()
or a page-local Stock load.
```

Do not activate this recovered pattern at module scope:

```
colossus_connection = colossus.connect("PROD")
```

Section 18 replaces it with a lazy `_real_colossus_connection()` function.

5. Make startup begin server-side after the cold page mounts

`StartupCoordinator.schedule_start()` already exists and is idempotent, but the page builders do not use it. Add a delayed server-side start as a fallback. Keep the existing browser interval and `/startz` paths; all paths converge on the same coordinator.

File: `ui/s09_factory.py`

Location: immediately after

```
app.server.config[STARTUP_COORDINATOR_CONFIG_KEY] = startup_coordinator
```

Add:

```
def schedule_cold_start() -> None:
    """Schedule one delayed writer after a cold financial page is requested."""
    if refresh_manager is None or startup_coordinator is None:
        return
    try:
        revision = int(refresh_manager.health.revision)
    except Exception:
        revision = 0
    if revision <= 0:
        startup_coordinator.schedule_start(delay_seconds=0.5)
```

Replace `cube_page_body()` **with**

```
def cube_page_body() -> html.Main:
    """Mount the revision-aware Risk page and schedule one cold writer."""
    schedule_cold_start()
    return html.Main(current_cube_page(), id="cube-page-container")
```

Do not make a partial edit to `pn1_page_body()` **here**

Section 11 contains the complete final replacement for `pn1_page_body()`. That replacement includes:

```
schedule_cold_start()
```

as its first executable line. Apply that complete replacement when you reach Section 11 rather than editing the same function twice.

Do not call `schedule_cold_start()` from Stock or Statics. Those pages should remain capable of mounting without triggering the entire financial refresh.

6. Build the shared hero in cold-loading mode immediately

File: `ui/s09_factory.py`

Function: `serve_layout()`

At the start of the function add:

```
shared_snapshot = current_shared_snapshot()
cold_start = (
    refresh_manager is not None
    and shared_snapshot is None
)
```

Replace the existing `build_shared_refresh_shell(...)` call with:

```
build_shared_refresh_shell(
    shared_snapshot,
    refresh_enabled=refresh_manager is not None,
    stage_delays=stage_delays,
    initial_loading=cold_start,
    keep_polling=cold_start,
    # A cold root page must paint the hero before the navigation callback runs.
    # Stock and Statics will hide it immediately through update_navigation().
    style={} if cold_start else {"display": "none"},
),
```

Do not call `current_shared_snapshot()` again inside this shell call.

7. Preserve the hero until the financial page consumes revision 1

The current callback uses the committed revision marker as though it were the page-rendered revision. Separate those values.

Replace the complete hydrate_shared_refresh_shell callback decorator and function with

```
@app.callback(
    Output("shared-refresh-shell", "children"),
    Input("initial-load-trigger", "n_intervals", allow_optional=True),
    Input("initial-load-retry", "n_clicks", allow_optional=True),
    Input("pnl-initial-load-trigger", "n_intervals", allow_optional=True),
    Input("shared-refresh-bootstrap-interval", "n_intervals"),
    State("refresh-status", "className", allow_optional=True),
    State("error-log", "children", allow_optional=True),
    State("data-revision-store", "data", allow_optional=True),
    State("refresh-commit-revision", "children", allow_optional=True),
    prevent_initial_call=True,
)
def hydrate_shared_refresh_shell(
    load_intervals,
    retry_clicks,
    pnl_intervals,
    _shared_intervals,
    status_class="",
    displayed_error="",
    displayed_data_revision=0,
    displayed_commit_revision=0,
):
    """Follow revision 1 independently of the currently mounted Dash page."""
    if (
        ctx.triggered_id == "shared-refresh-bootstrap-interval"
        and int(_shared_intervals or 0) <= 0
    ):
        raise PreventUpdate

    startup = start_or_follow_initial_snapshot(
        ctx.triggered_id,
        load_intervals,
        retry_clicks,
        pnl_intervals,
    )
    common_options = {
        "refresh_enabled": True,
        "stage_delays": refresh_manager.stage_delays,
    }

    if startup.phase == "succeeded" and refresh_manager.health.revision > 0:
        control = refresh_manager.control_snapshot
        try:
            rendered_revision = int(displayed_data_revision or 0)
        except (TypeError, ValueError):
            rendered_revision = 0
        try:
            commit_revision = int(displayed_commit_revision or 0)
        except (TypeError, ValueError):
            commit_revision = 0

        pending_handoff = rendered_revision < int(control.revision)
        if (
            not pending_handoff
            and commit_revision >= int(control.revision)
            and "is-refreshing" not in str(status_class or "").split()
        ):
            raise PreventUpdate

        return build_shared_refresh_shell(
            control,
            # The committed marker advances immediately. The live data Store
            # remains at the last page-rendered revision until JavaScript sees
            # a mounted warm Risk or P&L page and releases it.
            data_revision=rendered_revision,
            initial_loading=pending_handoff,
            keep_polling=pending_handoff,
            **common_options,
        ).children

    if startup.phase == "failed":
        error_text = startup.error or (
            "Initial data load failed. Check the server log and retry."
        )
        if (
            str(displayed_error or "") == str(error_text)
            and "is-error" in str(status_class or "").split()
        ):
            raise PreventUpdate
```

```

        raise PreventUpdate
    return build_shared_refresh_shell(
        None,
        initial_error=error_text,
        **common_options,
    ).children

if startup.phase == "stalled":
    if str(displayed_error or "") != str(startup.error or ""):
        return build_shared_refresh_shell(
            None,
            initial_error=startup.error,
            keep_polling=True,
            **common_options,
        ).children
    raise PreventUpdate

if "is-refreshing" not in str(status_class or "").split():
    return build_shared_refresh_shell(
        None,
        initial_loading=True,
        keep_polling=True,
        **common_options,
    ).children

raise PreventUpdate

```

8. Leave a completed hero visible long enough to paint

File: `assets/s02_app.js`

Function: `finishRefreshProgress()`

Replace:

```
}, hasNewError ? 5000 : 300);
```

with:

```
}, hasNewError ? 5000 : 1500);
```

This is presentation polish. The Python handoff changes above are the actual lifecycle fix.

Function: `recoverReadyBootstrap()`

Replace:

```
const handoffDeadline = Date.now() + 15000;
```

with:

```
const handoffDeadline = Date.now() + 60000;
```

Keep the existing session-scoped one-reload guard. This gives a large first React/Dash tree up to one minute to mount before the browser attempts its single recovery reload.

9. Add a targeted dashboard read so startup does not copy every large frame

The current `refresh_manager.snapshot` property deep-copies Risk status, Risk checker, combined P&L, MarketBook, dashboard, unmapped rows, and dictionaries. The cold Risk page only needs the compact control metadata and `dashboard_frame`.

File: `core/s02_pipeline.py`

Location: immediately after the existing `FrameRead` dataclass

Add:

```
@dataclass(frozen=True)
class DashboardRead:
    """One dashboard frame and compact controls from the same committed revision."""

    revision: int
    control: ControlSnapshot
    frame: pd.DataFrame
```

Inside `RiskRefreshManager`, replace the existing `control_snapshot` property with the following block and add `read_dashboard()` immediately below it

```
@staticmethod
def _control_from_snapshot(committed: RefreshSnapshot) -> ControlSnapshot:
    return ControlSnapshot(
        revision=committed.revision,
        refreshed_at=committed.refreshed_at,
        system_date=committed.system_date,
        market_date=committed.market_date,
        checker_date=committed.checker_date,
        market_status=committed.market_status,
        forced_view_date=committed.forced_view_date,
        risk_status=committed.risk_status.copy(deep=True),
        risk_checker_enabled=committed.risk_checker_enabled,
        commodity_market_enabled=committed.commodity_market_enabled,
        risk_dates=dict(committed.risk_dates),
        forced_dates=dict(committed.forced_dates),
        errors=committed.errors,
    )

@property
def control_snapshot(self) -> ControlSnapshot:
    """Return control metadata while copying only the readiness frame."""
    with self._state_lock:
        if self._snapshot is None:
            raise RuntimeError("RiskRefreshManager has not been refreshed yet")
        committed = self._snapshot
        return self._control_from_snapshot(committed)

def read_dashboard(self) -> DashboardRead:
    """Copy only the dashboard and control metadata from one snapshot object."""
    with self._state_lock:
        if self._snapshot is None:
            raise RuntimeError("RiskRefreshManager has not been refreshed yet")
        committed = self._snapshot
        return DashboardRead(
            revision=committed.revision,
            control=self._control_from_snapshot(committed),
            frame=committed.dashboard_frame.copy(deep=True),
        )
```

In `core/s02_pipeline.py::__all__` add

```
"DashboardRead",
```

File: ui/s01_contracts.py

Add after FrameReadProtocol

```
@runtime_checkable
class DashboardReadProtocol(Protocol):
    """One dashboard frame and compact control view from one revision."""

    @property
    def revision(self) -> int: ...

    @property
    def control(self) -> ControlSnapshotProtocol: ...

    @property
    def frame(self) -> pd.DataFrame: ...
```

In RefreshManagerProtocol add

```
def read_dashboard(self) -> DashboardReadProtocol: ...
```

In __all__ add

```
"DashboardReadProtocol",
```

File: ui/s09_factory.py

Delete these variables from build_app() :

```
risk_snapshot_lock = Lock()
risk_snapshot_revision = prepared_dashboard_revision
risk_snapshot_cache = initial_snapshot
```

Replace current_cube_page() with:

```
def current_cube_page():
    """Serve the shell cold and the complete dashboard after revision 1."""
    if refresh_manager is not None:
        try:
            if int(refresh_manager.health.revision) > 0:
                dashboard_read = refresh_manager.read_dashboard()
                prepared = prepared_committed_dashboard(
                    revision=int(dashboard_read.revision),
                    frame=dashboard_read.frame,
                )
                if prepared is None:
                    raise RuntimeError("Committed dashboard frame is unavailable")
                return build_layout(
                    prepared,
                    dashboard_read.control,
                    refresh_enabled=True,
                    stage_delays=stage_delays,
                    include_shared_refresh_shell=False,
                )
            except Exception as error:
                app.logger.exception(
                    "Could not materialize the committed startup dashboard: %s",
                    type(error).__name__,
                )
                return build_initial_load_layout(
                    stage_delays=stage_delays,
                    include_shared_refresh_shell=False,
                    error=(
                        "The validated data loaded, but the dashboard could not be "
                        "rendered. Check the server log and retry."
                    ),
                )
            return build_initial_load_layout(
                stage_delays=stage_delays,
                include_shared_refresh_shell=False,
            )
        return build_layout(
            risk_data,
            initial_snapshot,
            refresh_enabled=False,
            stage_delays=stage_delays,
            include_shared_refresh_shell=False,
```

```
)
```

File: ui/s07_events.py

Replace `materialize_initial_dashboard()` with:

```
def materialize_initial_dashboard() -> html.Div:
    """Build the full page from a targeted same-revision dashboard read."""
    try:
        dashboard_read = refresh_manager.read_dashboard()
        prepared = cache.replace_frame(
            dashboard_read.frame,
            dashboard_read.revision,
        )
        layout = build_layout(
            prepared,
            dashboard_read.control,
            refresh_enabled=True,
            stage_delays=refresh_manager.stage_delays,
            include_shared_refresh_shell=False,
        )
    except Exception as error:
        incident_id = uuid.uuid4().hex[:10]
        app.logger.error(
            "Cube startup UI preparation failed; incident=%s type=%s",
            incident_id,
            type(error).__name__,
            exc_info=True,
        )
        safe_error = (
            f"Dashboard preparation failed (incident {incident_id}). "
            "No dashboard was published; retry after checking the server log."
        )
        app.server.config[STARTUP_UI_ERROR_CONFIG_KEY] = safe_error
    return build_initial_load_layout(
        stage_delays=refresh_manager.stage_delays,
        error=safe_error,
        include_shared_refresh_shell=False,
    )
app.server.config[STARTUP_UI_ERROR_CONFIG_KEY] = None
return layout
```

In `load_initial_snapshot_after_first_paint()` , replace:

```
return materialize_initial_dashboard(refresh_manager.snapshot)
```

with:

```
return materialize_initial_dashboard()
```

File: ui/s04_components.py

Change the `build_layout()` parameter annotation from:

```
initial_snapshot: RefreshSnapshotProtocol | None = None,
```

into:

```
initial_snapshot: (
    ControlSnapshotProtocol | RefreshSnapshotProtocol | None
) = None,
```

Also change the first parameter annotation of `build_risk_date_editor()` from:

```
snapshot: RefreshSnapshotProtocol,
```

to:

```
snapshot: ControlSnapshotProtocol | RefreshSnapshotProtocol,
```

9.1 Reuse the factory's prepared dashboard cache inside Risk callbacks

Without this change, `prepared_committed_dashboard()` and `_RiskDataCache.replace_frame()` can both call `prepare_risk_data()` for the same committed revision. Use the factory cache as the one preparation authority.

File: `ui/s07_events.py`

Replace `_RiskDataCache.__init__()` with

```
def __init__(
    self,
    risk_data: pd.DataFrame,
    revision: int,
    *,
    prepared_frame_loader: Callable[... , pd.DataFrame | None] | None = None,
) -> None:
    self._lock = RLock()
    self._revision = int(revision)
    self._frame = risk_data
    self._prepared_frame_loader = prepared_frame_loader
    self._filtered: dict[tuple[Any, ...], pd.DataFrame] = {}
    self._rendered: OrderedDict[str, Any] = OrderedDict()
```

The module already imports `Callable` and `Any` .

Replace `_RiskDataCache.replace_frame()` completely

```
def replace_frame(
    self,
    frame: pd.DataFrame,
    revision: int,
) -> pd.DataFrame:
    """Publish one prepared dashboard frame to the callback cache."""
    selected_revision = int(revision)

    if self._prepared_frame_loader is None:
        prepared = prepare_risk_data(frame)
    else:
        prepared = self._prepared_frame_loader(
            revision=selected_revision,
            frame=frame,
        )
    if prepared is None:
        raise RuntimeError(
            "Prepared dashboard frame is unavailable"
        )

    with self._lock:
        if selected_revision <= self._revision:
            return self._frame

        self._frame = prepared
        self._revision = selected_revision
        self._filtered.clear()
        self._rendered.clear()
        return prepared
```

Add a parameter to `register_callbacks()`

Add after `market_history_loader` :

```
prepared_frame_loader: (
    Callable[... , pd.DataFrame | None] | None
) = None,
```

The end of the signature becomes:

```
def register_callbacks(
    app: Dash,
    refresh_manager: RefreshManagerProtocol | None,
    initial_snapshot: RefreshSnapshotProtocol | None,
    risk_data: pd.DataFrame,
    *,
    route_prefix: str = "/",
    startup_coordinator: StartupCoordinator | None = None,
```

```

market_history_loader: MarketHistoryLoaderProtocol | None = None,
prepared_frame_loader: (
    Callable[..., pd.DataFrame | None] | None
) = None,
) -> None:

```

Replace the `_RiskDataCache` construction

Replace:

```

cache = _RiskDataCache(
    risk_data,
    initial_snapshot.revision
    if initial_snapshot is not None
    else 0,
)

```

with:

```

cache = _RiskDataCache(
    risk_data,
    initial_snapshot.revision
    if initial_snapshot is not None
    else 0,
    prepared_frame_loader=prepared_frame_loader,
)

```

File: `ui/s09_factory.py`

Add the loader to the existing `register_callbacks()` call

Replace that complete call with:

```

register_callbacks(
    app,
    refresh_manager,
    initial_snapshot,
    risk_data,
    route_prefix=request_prefix,
    startup_coordinator=startup_coordinator,
    market_history_loader=market_history_loader,
    prepared_frame_loader=prepared_committed_dashboard,
)

```

After this change, the first caller that requests a revision prepares it; later Risk and P&L consumers receive the same process-local prepared frame.

10. Avoid the defensive full-snapshot copy that cold startup discards

The startup coordinator ignores the object returned by `RiskRefreshManager.refresh()`. Add an opt-out that retains the existing safe default for every normal caller.

File: `core/s02_pipeline.py`

In the `RiskRefreshManager.refresh()` signature add

```

copy_result: bool = True,

```

The end of the signature should look like:

```

def refresh(
    self,
    *,
    force_pl: bool = False,
    force_risk: bool = False,

```

```

commodity_market_enabled: bool = False,
risk_checker_enabled: bool | None = None,
forced_dates: Mapping[
    str,
    date | datetime | str | pd.Timestamp,
] | None = None,
view_date: date | datetime | str | pd.Timestamp | None = None,
reason: str = "status",
expected_revision: int | None = None,
copy_result: bool = True,
) -> RefreshSnapshot:

```

At the beginning of the method add:

```

if not isinstance(copy_result, bool):
    raise TypeError("copy_result must be boolean")

def release_result(snapshot: RefreshSnapshot) -> RefreshSnapshot:
    return self._copy_snapshot(snapshot) if copy_result else snapshot

```

Within `refresh()` only, replace every occurrence of:

```

return self._copy_snapshot(snapshot)

```

with:

```

return release_result(snapshot)

```

Do not change `refresh_portfolios()` in this step.

File: `ui/s01_contracts.py`

Add the optional parameter to the protocol method:

```

copy_result: bool = True,

```

File: `ui/s07_events.py`

Function: `StartupCoordinator._run()`

Replace the direct manager call with this compatibility-safe block:

```

refresh_kwargs: dict[str, object] = {
    "forced_dates": {},
    "view_date": None,
    "commodity_market_enabled": False,
    "risk_checker_enabled": True,
    "reason": "initial load",
    "expected_revision": 0,
}
try:
    parameters = signature(self._manager.refresh).parameters
except (TypeError, ValueError):
    parameters = {}
if "copy_result" in parameters:
    refresh_kwargs["copy_result"] = False

self._manager.refresh(**refresh_kwargs)

```

This removes one full copy of `combined_pl`, `market_frame`, `dashboard_frame`, `unmapped_frame`, `risk_checker`, and `risk_status` from the cold-start critical path.

11. Remove duplicate initial Risk and P&L table construction

File: `ui/s04_components.py`

Function: `build_layout()`

Keep:

```
initial_risk_type = risk_options[0]["value"]
top_book_open_rows = default_top_book_open_rows(risk_data)
```

Delete the entire block that creates:

```
initial_ir_family
initial_risk_frame
initial_open_rows
initial_risk_table
initial_aggregate_table
```

In the `aggregate-pl-grid` component, replace `initial_aggregate_table` with:

```
html.Div(
    "Loading Aggregate P&L...",
    className="empty-state",
    role="status",
)
```

The block should become:

```
html.Div(
    dcc.Loading(
        html.Div(
            html.Div(
                "Loading Aggregate P&L...",
                className="empty-state",
                role="status",
            ),
            id="aggregate-pl-grid",
        ),
        custom_spinner=build_cube_loader("Loading aggregate P&L"),
        delay_show=120,
        className="cube-loading-boundary",
    ),
    className="aggregate-pl-panel",
),
```

In `risk-grid`, replace `initial_risk_table` with:

```
html.Div(
    "Loading Risk Explorer...",
    className="empty-state",
    role="status",
)
```

The block should become:

```
html.Div(
    html.Div(
        html.Div(
            "Loading Risk Explorer...",
            className="empty-state",
            role="status",
        ),
        id="risk-grid",
        className="risk-grid",
    ),
    id="main-risk-panel",
    className="risk-panel",
),
```

The existing mounted callbacks remain the sole builders of the two expensive tables.

File: ui/s09_factory.py

Replace `pnl_page_body()` with:

```
def pnl_page_body():
    """Mount Aggregate P&L and the optional sender on the native P&L route."""
    schedule_cold_start()
    if refresh_manager is not None:
        try:
            start_initial_load = int(refresh_manager.health.revision) <= 0
        except Exception:
            start_initial_load = True
        app.logger.exception("Could not read the P&L startup revision")

    return build_pl_page(
        start_initial_load=start_initial_load,
        send_workflow_available=pl_send_config is not None,
        initial_aggregate_frame=None,
        saved_view_bar=build_saved_filter_view_bar(
            PL_SAVED_VIEW_CONTROLS,
            filter_note=PL_FILTER_NOTE,
            filter_bar=build_pl_filter_bar(None),
        ),
    )

return html.Main(
    [
        html.H1("P&L Sender", className="static-data-page-title"),
        html.P(
            "P&L sending is not configured for this application.",
            id="pnl-unavailable",
            className="static-data-empty",
        ),
    ],
    id="pnl-page",
    className="static-data-page",
)
```

Do not delete `prepared_committed_dashboard()` . Other callbacks still use its revision cache.

12. Remove the intentional one-second Risk product delay

File: s01_app.py

Replace:

```
"risk_product": float(os.getenv("RISK_PRODUCT_DELAY_SECONDS", "1")),
```

with:

```
"risk_product": float(os.getenv("RISK_PRODUCT_DELAY_SECONDS", "0")),
```

This delay is not applied on the first cold snapshot, but it unnecessarily slows every later Risk reload and date-driven Risk refresh.

13. Add bounded connector task workers inside the one process

This is the correct place to use multiple workers for real market data and the active fake synchronous connector path.

The same manager methods call whichever adapter is registered:

- fake CSV connectors in the checked-in runtime; or
- real product adapters after the registration switch.

Therefore this change exercises the same concurrency path in both environments.

Keep the checked-in fake connector deterministic:

```
CUBE_RISK_CONNECTOR_WORKERS=1
CUBE_MARKET_CONNECTOR_WORKERS=1
CUBE_CONNECTOR_BATCH_TIMEOUT_SECONDS=300
```

After activating the real adapters and confirming client thread safety and API rate limits, start with:

```
CUBE_RISK_CONNECTOR_WORKERS=2
CUBE_MARKET_CONNECTOR_WORKERS=4
```

Then benchmark Risk 4 and Market 8 , changing one value at a time.

File: `core/s02_pipeline.py`

Add these imports near the top

```
from concurrent.futures import (
    Future,
    ThreadPoolExecutor,
    TimeoutError as FuturesTimeoutError,
    as_completed,
)
```

Add these parameters to `RiskRefreshManager.__init__()`

Place them after `stage_delays` :

```
risk_workers: int = 1,
market_workers: int = 1,
connector_batch_timeout_seconds: float = 300.0,
```

Add this validation inside `__init__()` before assigning instance fields

```
for name, value in (
    ("risk_workers", risk_workers),
    ("market_workers", market_workers),
):
    if isinstance(value, (bool, np.bool_)) or not isinstance(value, int):
        raise TypeError(f"{name} must be a positive integer")
    if value <= 0:
        raise ValueError(f"{name} must be greater than zero")

if isinstance(
    connector_batch_timeout_seconds,
    (bool, np.bool_),
) or not isinstance(connector_batch_timeout_seconds, Real):
    raise TypeError(
        "connector_batch_timeout_seconds must be a positive real number"
    )
connector_timeout = float(connector_batch_timeout_seconds)
if not np.isfinite(connector_timeout) or connector_timeout <= 0:
    raise ValueError(
        "connector_batch_timeout_seconds must be a positive finite number"
    )
```

Add these assignments after the existing loader assignments

```
self._risk_workers = int(risk_workers)
self._market_workers = int(market_workers)
self._connector_batch_timeout_seconds = connector_timeout
self._risk_executor = ThreadPoolExecutor(
    max_workers=self._risk_workers,
    thread_name_prefix="cube-risk-source",
)
self._market_executor = ThreadPoolExecutor(
    max_workers=self._market_workers,
    thread_name_prefix="cube-market-source",
)
```

The executors are bounded and process-local. Their threads are created lazily when the first task is submitted.

Add this method inside RiskRefreshManager , immediately before `_load_product_risk()`

```
def _collect_connector_frames(
    self,
    future_map: Mapping[Future, tuple[int, str]],
    *,
    function_name: str,
    stage: str,
    source_type: str | None,
    unit_name: str,
) -> list[pd.DataFrame]:
    """Collect one bounded connector batch in deterministic input order."""
    if not future_map:
        return []

    ordered: list[pd.DataFrame | None] = [None] * len(future_map)
    try:
        completed = 0
        for future in as_completed(
            tuple(future_map),
            timeout=self._connector_batch_timeout_seconds,
        ):
            position, label = future_map[future]
            frame = future.result()
            if not isinstance(frame, pd.DataFrame):
                raise TypeError(
                    f"{function_name} returned a non-DataFrame for {label!r}"
                )
            ordered[position] = frame
            completed += 1

            progress_values: dict[str, object] = {}
            if unit_name == "Underlying":
                progress_values["underlying"] = label
            else:
                progress_values["product_label"] = label

            self._progress_activity(
                function_name,
                stage,
                source_type=source_type,
                product_index=completed,
                product_total=len(future_map),
                message=(
                    f"Completed {unit_name} {completed} of "
                    f"{len(future_map)}: {label}."
                ),
                **progress_values,
            )
    except FuturesTimeoutError as exc:
        pending_labels = [
            label
            for future, (_position, label) in future_map.items()
            if not future.done()
        ]
        for future in future_map:
            future.cancel()
        self._progress_activity(
            function_name,
            stage,
            source_type=source_type,
            message=(
                f"Connector batch exceeded "
                f"{self._connector_batch_timeout_seconds:g}s; "
                f"pending={pending_labels[:10]}."
            ),
        )
        raise TimeoutError(
            f"{function_name} exceeded the connector batch timeout; "
            f"pending={pending_labels[:10]}"
        ) from exc
    except Exception:
        for future in future_map:
            future.cancel()
        raise

    if any(frame is None for frame in ordered):
        raise RuntimeError(f"{function_name} did not complete every submitted task")
    return [frame for frame in ordered if frame is not None]
```

A Python thread cannot forcibly terminate an arbitrary blocked synchronous library call. The batch timeout makes the refresh fail visibly and cancels queued tasks, but every real client must still have its own

socket/request/database timeout configured. The executor is persistent and bounded, so a blocked call cannot create an unbounded number of new framework threads.

14. Parallelize independent Risk product source calls

Keep fake mode at `CUBE_RISK_CONNECTOR_WORKERS=1` . In real mode begin with `2` . Set it back to `1` if the private client library uses a shared mutable session.

File: `core/s02_pipeline.py`

Replace `_load_product_risk()` with

```
def _load_product_risk(
    self,
    spec: ProductSpec,
    risk_date: pd.Timestamp,
    *,
    product_index: int = 0,
    product_total: int = 0,
) -> pd.DataFrame:
    """Load one product Risk frame through its registered connector."""
    adapter = self._connector_adapters.get(spec.source_type)
    connector = adapter.risk if adapter is not None else self._risk_loader
    self._progress_step(
        _callable_name(connector),
        "risk",
        source_type=spec.source_type,
        product_label=_product_progress_label(spec),
        product_index=product_index,
        product_total=product_total,
        message="Loading connector Risk/dRisk.",
    )
    if adapter is not None:
        return adapter.risk(risk_date)
    return self._risk_loader(risk_date, spec.source_type)
```

In `RiskRefreshManager.refresh()` , find the block beginning with

```
for product_index, spec in enumerate(risk_specs, start=1):
```

Replace that complete Risk loading/validation loop with:

```
raw_risk_by_source: dict[str, pd.DataFrame] = {}
if risk_specs:
    if self._risk_workers == 1 or len(risk_specs) == 1:
        for product_index, spec in enumerate(risk_specs, start=1):
            raw_risk_by_source[spec.source_type] = self._load_product_risk(
                spec,
                next_dates[spec.source_type],
                product_index=product_index,
                product_total=len(risk_specs),
            )
    else:
        future_map: dict[Future, tuple[int, str]] = {}
        for position, spec in enumerate(risk_specs):
            future = self._risk_executor.submit(
                self._load_product_risk,
                spec,
                next_dates[spec.source_type],
                product_index=position + 1,
                product_total=len(risk_specs),
            )
            future_map[future] = (
                position,
                _product_progress_label(spec),
            )

        raw_frames = self._collect_connector_frames(
            future_map,
            function_name="get_product_risk_connectors",
            stage="risk",
            source_type=None,
            unit_name="Product",
```

```

    )
    raw_risk_by_source = {
        spec.source_type: raw_frames[position]
        for position, spec in enumerate(risk_specs)
    }

for product_index, spec in enumerate(risk_specs, start=1):
    source_type = spec.source_type
    product_label = _product_progress_label(spec)
    self._progress_step(
        f"get_{spec.key}_risk",
        "risk",
        source_type=source_type,
        product_label=product_label,
        product_index=product_index,
        product_total=len(risk_specs),
        hold_seconds=risk_product_delay,
        message=f"Validating Risk/dRisk for {product_label}.",
    )
    next_risk[source_type] = get_product_risk(
        spec,
        next_dates[source_type],
        raw_risk_by_source[source_type],
    )
    if risk_product_delay > 0:
        self._sleep(risk_product_delay)

```

Validation and publication remain deterministic and sequential. Only independent source I/O overlaps.

15. Parallelize real or fake per-Underlying market calls

This is likely the largest cold-start source-I/O improvement. The existing code makes one Open call and one Current call for every Underlying in sequence.

File: core/s02_pipeline.py

Replace `_load_product_market_open()` with

```
def _load_product_market_open(
    self,
    spec: ProductSpec,
    open_date: pd.Timestamp,
    underlyings: tuple[str, ...],
    *,
    market_status: str,
) -> pd.DataFrame:
    """Load one product's Open quotes with bounded Underlying concurrency."""
    adapter = self._connector_adapters.get(spec.source_type)
    connector = (
        adapter.market_open
        if adapter is not None
        else self._market_open_loader
    )
    selected_status = _require_market_status(market_status)
    requested = tuple(underlyings)
    if not requested:
        return pd.DataFrame()

    def load_one(underlying: str) -> pd.DataFrame:
        frame = (
            adapter.market_open(
                open_date,
                underlying,
                market_status=selected_status,
            )
            if adapter is not None
            else self._market_open_loader(
                spec.source_type,
                open_date,
                underlying,
                market_status=selected_status,
            )
        )
        if not isinstance(frame, pd.DataFrame):
            raise TypeError(
                "market Open connector must return a pandas DataFrame"
            )
        return frame

    if self._market_workers == 1 or len(requested) == 1:
        frames: list[pd.DataFrame] = []
        for completed, underlying in enumerate(requested, start=1):
            self._progress_activity(
                _callable_name(connector),
                "market_open",
                source_type=spec.source_type,
                underlying=underlying,
                product_index=completed,
                product_total=len(requested),
                message=f"Loading Open for {underlying}.",
            )
            frames.append(load_one(underlying))
        return pd.concat(frames, ignore_index=True, sort=False)

    future_map: dict[Future, tuple[int, str]] = {}
    for position, underlying in enumerate(requested):
        future = self._market_executor.submit(load_one, underlying)
        future_map[future] = (position, underlying)

    frames = self._collect_connector_frames(
        future_map,
        function_name=_callable_name(connector),
        stage="market_open",
        source_type=spec.source_type,
        unit_name="Underlying",
    )
    return pd.concat(frames, ignore_index=True, sort=False)
```

Replace `_load_product_market_status()` with

```
def _load_product_market_status(
    self,
    spec: ProductSpec,
    market_date: pd.Timestamp,
    underlyings: tuple[str, ...],
    *,
    market_status: str,
) -> pd.DataFrame:
    """Load one product's Current quotes with bounded Underlying concurrency."""
    adapter = self._connector_adapters.get(spec.source_type)
    connector = (
        adapter.market_status
        if adapter is not None
        else self._market_status_loader
    )
    selected_status = _require_market_status(market_status)
    requested = tuple(underlyings)
    if not requested:
        return pd.DataFrame()

    def load_one(underlying: str) -> pd.DataFrame:
        frame = (
            adapter.market_status(
                market_date,
                underlying,
                market_status=selected_status,
            )
            if adapter is not None
            else self._market_status_loader(
                spec.source_type,
                market_date,
                underlying,
                market_status=selected_status,
            )
        )
        if not isinstance(frame, pd.DataFrame):
            raise TypeError(
                "current market connector must return a pandas DataFrame"
            )
        return frame

    if self._market_workers == 1 or len(requested) == 1:
        frames: list[pd.DataFrame] = []
        for completed, underlying in enumerate(requested, start=1):
            self._progress_activity(
                _callable_name(connector),
                "market_status",
                source_type=spec.source_type,
                underlying=underlying,
                product_index=completed,
                product_total=len(requested),
                message=f>Loading {selected_status} for {underlying}.",
            )
            frames.append(load_one(underlying))
        return pd.concat(frames, ignore_index=True, sort=False)

    future_map: dict[Future, tuple[int, str]] = {}
    for position, underlying in enumerate(requested):
        future = self._market_executor.submit(load_one, underlying)
        future_map[future] = (position, underlying)

    frames = self._collect_connector_frames(
        future_map,
        function_name=_callable_name(connector),
        stage="market_status",
        source_type=spec.source_type,
        unit_name="Underlying",
    )
    return pd.concat(frames, ignore_index=True, sort=False)
```

The returned concatenation still follows the original requested Underlying order, regardless of completion order.

16. Pass connector worker settings from the feed composition root

File: `feeds/s01_sources.py`

Add these helpers immediately above `build_production_refresh_manager()`

```
def _positive_int_env(name: str, default: int) -> int:
    raw = os.getenv(name, str(default))
    try:
        value = int(raw)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"{name} must be a positive integer") from exc
    if value <= 0:
        raise ValueError(f"{name} must be greater than zero")
    return value

def _positive_float_env(name: str, default: float) -> float:
    raw = os.getenv(name, str(default))
    try:
        value = float(raw)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"{name} must be a positive number") from exc
    if not np.isfinite(value) or value <= 0:
        raise ValueError(f"{name} must be a positive finite number")
    return value
```

Because `_positive_float_env` uses NumPy, add this active import near the top:

```
import numpy as np
```

Inside `build_production_refresh_manager()` add

```
risk_workers = _positive_int_env(
    "CUBE_RISK_CONNECTOR_WORKERS",
    1,
)
market_workers = _positive_int_env(
    "CUBE_MARKET_CONNECTOR_WORKERS",
    1,
)
connector_timeout = _positive_float_env(
    "CUBE_CONNECTOR_BATCH_TIMEOUT_SECONDS",
    300.0,
)
```

Add these arguments to the `RiskRefreshManager(...)` call

```
risk_workers=risk_workers,
market_workers=market_workers,
connector_batch_timeout_seconds=connector_timeout,
```

The final call should contain:

```
return RiskRefreshManager(
    get_portfolio_config,
    thresholds=get_risk_thresholds,
    reported_underlyings=get_reported_underlyings,
    risk_checker_loader=get_risk_checker,
    market_status_resolver=resolve_market_state,
    risk_loader=get_risk,
    cross_gamma_matrix_loader=get_cross_gamma_sensitivities,
    new_trades_loader=get_new_trades,
    market_open_loader=get_market_open,
    market_status_loader=get_market_status,
    connector_adapters=get_product_connector_adapters(),
    stage_delays=stage_delays,
    trading_timezone=trading_timezone,
    risk_workers=risk_workers,
    market_workers=market_workers,
    connector_batch_timeout_seconds=connector_timeout,
)
```

17. Replace the recovered single-worker async bridge

The recovered `run_async` uses `ThreadPoolExecutor(max_workers=1)`. If the real adapters are enabled unchanged, all qcd/RAMP coroutines pass through one serialized thread. Replace it with one lazily started event loop that accepts many thread-safe coroutine submissions.

File: `adapters/s01_common.py`

Keep the existing active schema helper functions, but add the following active imports after `from __future__ import annotations`:

```
import asyncio
import os
from concurrent.futures import TimeoutError as FutureTimeoutError
from threading import Event, Lock, Thread
from typing import Any, Protocol
```

Remove the existing active line:

```
from typing import Protocol
```

because `Protocol` is imported in the combined import above.

Add this block before class RiskSource

```
_ASYNC_LOOP_LOCK = Lock()
_ASYNC_LOOP: asyncio.AbstractEventLoop | None = None
_ASYNC_LOOP_THREAD: Thread | None = None

def _async_timeout_seconds() -> float:
    raw = os.getenv(
        "CUBE_ASYNC_CONNECTOR_TIMEOUT_SECONDS",
        "120",
    )
    try:
        timeout = float(raw)
    except (TypeError, ValueError) as exc:
        raise ValueError(
            "CUBE_ASYNC_CONNECTOR_TIMEOUT_SECONDS must be a positive number"
        ) from exc
    if timeout <= 0:
        raise ValueError(
            "CUBE_ASYNC_CONNECTOR_TIMEOUT_SECONDS must be greater than zero"
        )
    return timeout

def _ensure_async_loop() -> asyncio.AbstractEventLoop:
    """Start one daemon event-loop thread only when a real adapter needs it."""
    global _ASYNC_LOOP, _ASYNC_LOOP_THREAD

    with _ASYNC_LOOP_LOCK:
        if (
            _ASYNC_LOOP is not None
            and not _ASYNC_LOOP.is_closed()
            and _ASYNC_LOOP_THREAD is not None
            and _ASYNC_LOOP_THREAD.is_alive()
        ):
            return _ASYNC_LOOP

        loop = asyncio.new_event_loop()
        ready = Event()

        def run_loop() -> None:
            asyncio.set_event_loop(loop)
            ready.set()
            loop.run_forever()

        thread = Thread(
            target=run_loop,
            name="cube-async-connectors",
            daemon=True,
        )
        thread.start()
        if not ready.wait(timeout=5.0):
            raise RuntimeError("Cube async connector loop did not start")

        _ASYNC_LOOP = loop
        _ASYNC_LOOP_THREAD = thread
        return loop

def run_async(
    coro: Any,
    *,
    timeout_seconds: float | None = None,
) -> Any:
    """Submit one coroutine to the shared connector event loop."""
    if not asyncio.iscoroutine(coro):
        raise TypeError("run_async expects a coroutine object")

    timeout = (
        _async_timeout_seconds()
        if timeout_seconds is None
        else float(timeout_seconds)
    )
    if timeout <= 0:
        raise ValueError("timeout_seconds must be greater than zero")

    future = asyncio.run_coroutine_threadsafe(
        coro,
        _ensure_async_loop(),
    )
    try:
        return future.result(timeout=timeout)
    except FutureTimeoutError as exc:
        future.cancel()
```

```
raise TimeoutError(
    f"Async connector exceeded {timeout:g} seconds"
) from exc
```

Add `run_async` to `__all__`

```
__all__ = [
    "MarketSource",
    "RiskSource",
    "exact_frame",
    "exact_status",
    "exact_underlying",
    "market_frame",
    "run_async",
]
```

Why this design:

- the manager's market task threads can call `run_async` concurrently;
- all coroutines are scheduled thread-safely on one event loop;
- while one coroutine awaits network I/O, the loop can run the others;
- each wait has a finite timeout;
- no nested submission waits on the same one-worker executor.

A coroutine must still use genuinely asynchronous I/O. CPU-heavy work inside an `async def` function will block this event loop and must remain outside it.

18. Parallelize the three independent real Portfolio sources

The recovered Portfolio mapping fetches Colossus, ConfigManager, and MRX sequentially. Fetch them concurrently, then merge them in one deterministic main-thread step.

The checked-in fake CSV path remains available and uses the same public function.

File: `feeds/s01_sources.py`

Add these imports

```
from concurrent.futures import ALL_COMPLETED, ThreadPoolExecutor, wait
from threading import RLock
```

The file already imports `lru_cache` .

Extend the `core.s01_schema` import with

```
PORTFOLIO_COLUMN,
PORTFOLIO_CONFIG_COLUMNS,
UNSPECIFIED_VALUE,
```

Add the following complete implementation immediately above `get_portfolio_config()`

```
def _env_enabled(name: str, default: bool = False) -> bool:
    raw = os.getenv(name)
    if raw is None:
        return default
    return raw.strip().casefold() in {"1", "true", "yes", "on"}

_REAL_COLOSSUS_REQUEST_LOCK = RLock()

@lru_cache(maxsize=1)
def _real_colossus_connection():
    """Create the real connection lazily so fixture startup imports no client."""
    import colossus

    return colossus.connect("PROD")

def _normalize_portfolio_column(frame: pd.DataFrame) -> pd.DataFrame:
    result = frame.copy()
    result[PORTFOLIO_COLUMN] = (
        result[PORTFOLIO_COLUMN]
        .astype(str)
        .str.replace(",", "", regex=False)
        .str.replace(r"\.0$", "", regex=True)
        .str.strip()
    )
    return result

def _load_colossus_portfolio_source(
    portfolio_date: pd.Timestamp,
) -> pd.DataFrame:
    body = {
        "reportingCurrency": "EUR",
        "attributes": ["Ptf", "PtfName", "SignoffGroup"],
        "values": ["SteppedDailyPnL"],
        "filters": [
            {
                "attributeName": "PnlType",
                "filters": ["Gross P&L"],
            }
        ],
        "fromDateKey": (
            portfolio_date - pd.offsets.BDay(2)
        ).strftime("%Y%m%d"),
        "toDateKey": portfolio_date.strftime("%Y%m%d"),
    }

    # Configure a finite HTTP timeout on the site-owned Colossus client or
    # connection object. Do not leave the network layer with an infinite wait.
    with _REAL_COLOSSUS_REQUEST_LOCK:
        response = _real_colossus_connection().raw_request(
            "POST",
            endpoint_uri="/v1/data-warehouse/reports/query",
            body=body,
        )
    frame = pd.DataFrame(
        [row["fields"] for row in response["results"][1:]],
        columns=response["results"][0]["fields"],
    )
    frame = frame.drop(
        columns=["SteppedDailyPnL"],
        errors="ignore",
    ).rename(
        columns={
            "Ptf": PORTFOLIO_COLUMN,
            "PtfName": "Portfolio Name",
        }
    )
    return _normalize_portfolio_column(frame)

def _load_configmanager_portfolio_source() -> pd.DataFrame:
    from awacs_poc import configmanager as cm

    frame = pd.DataFrame(
        cm.get("XVA.IM Optin.PnL.Ann.Ptf List")
    )
    if PORTFOLIO_COLUMN not in frame:
        raise ValueError(
            "ConfigManager portfolio source is missing 'Portfolio'"
        )
```

```

    )
    frame[PORTFOLIO_COLUMN] = (
        frame[PORTFOLIO_COLUMN]
        .fillna(UNSPECIFIED_VALUE)
        .astype(str)
        .str.replace(" ", "", regex=False)
    )
    return _normalize_portfolio_column(frame)

def _load_mrx_product_source(
    portfolio_date: pd.Timestamp,
) -> pd.DataFrame:
    import mrx

    view = mrx.MRXView("mrx/static/product.tsv")
    view += (
        "Current Date",
        portfolio_date.strftime("%Y/%m/%d"),
    )
    product_data = view.fetch(verify=False)

    xva_products = {
        "CVA-NONRISKMANAGED",
        "CVA-RISKMANAGED",
        "FCVA",
        "FBVA",
        "COLVA",
    }
    portfolio_product = product_data[
        [PORTFOLIO_COLUMN, "Product"]
    ].drop_duplicates()
    portfolio_xva = portfolio_product.loc[
        portfolio_product["Product"].isin(xva_products)
    ].drop_duplicates(subset=PORTFOLIO_COLUMN)
    portfolio_xva = portfolio_xva.copy()
    portfolio_xva["Product"] = "XVA"

    all_portfolios = product_data[
        PORTFOLIO_COLUMN
    ].drop_duplicates()
    portfolio_hedges = all_portfolios.loc[
        ~all_portfolios.isin(
            portfolio_xva[PORTFOLIO_COLUMN]
        )
    ].to_frame()
    portfolio_hedges["Product"] = "Hedges"

    result = pd.concat(
        [
            portfolio_xva[[PORTFOLIO_COLUMN, "Product"]],
            portfolio_hedges[[PORTFOLIO_COLUMN, "Product"]],
        ],
        ignore_index=True,
    )
    return _normalize_portfolio_column(result)

def _merge_real_portfolio_sources(
    ptf_mapping: pd.DataFrame,
    sog_colossus: pd.DataFrame,
    product_map: pd.DataFrame,
) -> pd.DataFrame:
    ptf_mapping = _normalize_portfolio_column(ptf_mapping)
    sog_colossus = _normalize_portfolio_column(sog_colossus)
    product_map = _normalize_portfolio_column(product_map)

    cit_override = sog_colossus.loc[
        sog_colossus["SignoffGroup"].eq("CIT XVA"),
        [PORTFOLIO_COLUMN, "Portfolio Name"],
    ].copy()
    cit_override["Product"] = np.where(
        cit_override["Portfolio Name"].astype(str).str.contains(
            "HED",
            na=False,
        ),
        "Hedges",
        "XVA",
    )
    product_map = pd.concat(
        [
            product_map,
            cit_override[[PORTFOLIO_COLUMN, "Product"]],
        ],
        ignore_index=True,
    ).drop_duplicates(PORTFOLIO_COLUMN, keep="last")

```

```

frame = ptf_mapping.merge(
    sog_colossus,
    on=PORTFOLIO_COLUMN,
    how="outer",
    suffixes=("", "_colossus"),
)

cit_name_rows = frame.loc[
    frame.get(
        "SignoffGroup",
        pd.Series("", index=frame.index),
    ).eq("CIT XVA")
].copy()
if not cit_name_rows.empty and "Portfolio Name" in cit_name_rows:
    cit_name_rows[PORTFOLIO_COLUMN] = cit_name_rows[
        "Portfolio Name"
    ].astype(str)
    frame = pd.concat(
        [frame, cit_name_rows],
        ignore_index=True,
    )

frame = frame.merge(
    product_map,
    on=PORTFOLIO_COLUMN,
    how="outer",
)

for column in PORTFOLIO_CONFIG_COLUMNS:
    if column not in frame:
        frame[column] = UNSPECIFIED_VALUE
    frame[column] = (
        frame[column]
        .fillna(UNSPECIFIED_VALUE)
        .astype(str)
        .str.strip()
        .replace("", UNSPECIFIED_VALUE)
    )

frame = frame.loc[
    frame["Product"].isin(["XVA", "Hedges"])
]
return (
    frame.loc[:, list(PORTFOLIO_CONFIG_COLUMNS)]
    .drop_duplicates(PORTFOLIO_COLUMN, keep="last")
    .reset_index(drop=True)
)

@lru_cache(maxsize=16)
def _load_real_portfolio_config_cached(
    portfolio_date_text: str,
) -> pd.DataFrame:
    portfolio_date = pd.Timestamp(
        portfolio_date_text
    ).normalize()
    timeout = _positive_float_env(
        "CUBE_PORTFOLIO_BATCH_TIMEOUT_SECONDS",
        180.0,
    )

    executor = ThreadPoolExecutor(
        max_workers=3,
        thread_name_prefix="cube-portfolio-source",
    )
    futures = {
        "colossus": executor.submit(
            _load_colossus_portfolio_source,
            portfolio_date,
        ),
        "configmanager": executor.submit(
            _load_configmanager_portfolio_source,
        ),
        "mrx": executor.submit(
            _load_mrx_product_source,
            portfolio_date,
        ),
    }
    done, pending = wait(
        tuple(futures.values()),
        timeout=timeout,
        return_when=ALL_COMPLETED,
    )
    if pending:
        pending_names = [

```

```

        name
        for name, future in futures.items()
        if future in pending
    ]
    for future in pending:
        future.cancel()
    executor.shutdown(
        wait=False,
        cancel_futures=True,
    )
    raise TimeoutError(
        "Portfolio source batch exceeded "
        f"{timeout:g}s; pending={pending_names}"
    )

try:
    results = {
        name: future.result()
        for name, future in futures.items()
    }
finally:
    executor.shutdown(wait=True)

return _merge_real_portfolio_sources(
    results["configmanager"],
    results["colossus"],
    results["mrx"],
)

def clear_portfolio_config_cache() -> None:
    _load_real_portfolio_config_cached.cache_clear()
    _real_colossus_connection.cache_clear()

```

Replace the executable body of `get_portfolio_config()` with

```

_business_date(
    portfolio_date,
    parameter="portfolio_date",
)
selected_date = pd.Timestamp(portfolio_date).normalize()

if _env_enabled("CUBE_USE_REAL_PORTFOLIO_CONFIG"):
    return _load_real_portfolio_config_cached(
        selected_date.date().isoformat()
    ).copy(deep=True)

frame = _read_fake_csv("portfolio_config")
_require_fake_notice(
    frame,
    [
        column
        for column in PORTFOLIO_CONFIG_REQUIRED_COLUMNS
        if column
        != PORTFOLIO_FIELD_BY_KEY["product"].external_name
    ],
    dataset="portfolio_config",
)
return frame.copy(deep=True)

```

Immediately after the function definition add:

```

get_portfolio_config.cache_clear = ( # type: ignore[attr-defined]
    clear_portfolio_config_cache
)

```

Configure real-client I/O timeouts

The framework batch timeout prevents the refresh from waiting forever at the orchestration level, but a running Python thread cannot be killed safely. Configure finite timeouts in the actual private clients as well:

- Colossus HTTP request timeout.
- ConfigManager request timeout.
- MRX fetch timeout.
- qcd/RAMP coroutine timeout through `run_async` .

Do not leave any remote call with an infinite socket wait.

19. Clear the Portfolio cache only for an explicit Portfolio/force refresh

A date-keyed cache avoids refetching the same governance during normal callbacks and Stock loads. The Refresh Portfolios button must still bypass it.

File: `core/s02_pipeline.py`

Add this module helper near `_load_portfolio_config()`

```
def _clear_source_cache(source: object) -> None:
    """Clear an optional connector-owned cache without importing its module."""
    cache_clear = getattr(source, "cache_clear", None)
    if callable(cache_clear):
        cache_clear()
```

In `refresh_portfolios()` , immediately before `_load_portfolio_config(...)` , add

```
_clear_source_cache(self._config_source)
```

The block should become:

```
_clear_source_cache(self._config_source)
next_config = _load_portfolio_config(
    self._config_source,
    portfolio_date,
)
```

In `refresh()` , within the final block, replace

```
if force_risk or market_date_changed or base_config is None:
    portfolio_date = checker_date
    next_config = _load_portfolio_config(
        self._config_source, portfolio_date
    )
```

with:

```
if force_risk or market_date_changed or base_config is None:
    portfolio_date = checker_date
    if force_risk:
        _clear_source_cache(self._config_source)
    next_config = _load_portfolio_config(
        self._config_source,
        portfolio_date,
    )
```

20. Do not add generic “async CSV reads”

The active fixture path already caches parsed files and per-Source/Underlying partitions. `pandas.read_csv()` is blocking; wrapping it in `async def` does not make it asynchronous.

The manager-level connector task pool above already allows the synchronous fake connector functions to run through the same bounded task path as the real connectors. That is sufficient for integration testing.

Only optimize the remaining repeated `Path.stat()` calls if profiling shows the files are on slow network storage. On a local filesystem this is below the priority of Portfolio collapse, source concurrency, snapshot copying, and duplicate rendering.

21. Collapse Portfolio before publishing dashboard_frame

This is the primary row-count reduction. Keep Portfolio in raw connectors, combined_pl , mapping, diagnostics, Stock enrichment, and Portfolio P&L sending. Remove it only from the analytical frame published to Dash.

File: core/s02_pipeline.py

Replace the complete `to_dashboard_frame()` function with

```
def _sum_available(values: pd.Series):
    """Sum populated values while preserving an all-missing group as missing."""
    return values.sum(min_count=1)

def to_dashboard_frame(frame: pd.DataFrame) -> pd.DataFrame:
    """Publish one Portfolio-free analytical projection to Dash."""
    _require_columns(
        frame,
        [
            SOURCE_TYPE,
            RISK_TYPE,
            RISK_GREEK,
            SPLIT,
            *PORTFOLIO_METADATA_COLUMNS,
            PORTFOLIO_MAPPED,
            DISPLAY_BUCKET,
            GROUP,
            UNDERLYING,
            REPORTED_UNDERLYING,
            *TENOR_COLUMNS,
            *TENOR_ORDER_COLUMNS,
            PORTFOLIO,
            RISK,
            DRISK,
            OPEN,
            CURRENT,
            PL,
            MARKET_MOVE,
            MARKET_AVAILABLE,
            MARKET_DATA_STATUS,
            PROMOTION_REASON,
            PROMOTION_SCORE,
            RISK_THRESHOLD,
            DRISK_THRESHOLD,
            PL_THRESHOLD,
        ],
        "combined P&L",
    )

    credit_columns = [
        column
        for column in CREDIT_MEASURE_COLUMNS
        if column in frame
    ]
    additive_columns = [
        RISK,
        DRISK,
        PL,
        *credit_columns,
    ]
    group_columns = [
        SOURCE_TYPE,
        RISK_TYPE,
        RISK_GREEK,
        SPLIT,
        *PORTFOLIO_POSITION_COLUMNS,
        GROUP,
        *([REGION] if REGION in frame else []),
        REPORTED_UNDERLYING,
        UNDERLYING,
        *TENOR_COLUMNS,
        *PORTFOLIO_REPORTING_COLUMNS,
        PORTFOLIO_MAPPED,
    ]

    aggregations: dict[str, object] = {
        **{
            column: _sum_available
            for column in additive_columns
        },
        **{
            column: "min"
            for column in TENOR_ORDER_COLUMNS
        },
        DISPLAY_BUCKET: "first",
        PROMOTION_REASON: "first",
        PROMOTION_SCORE: "max",
        RISK_THRESHOLD: "first",
        DRISK_THRESHOLD: "first",
    }
```

```

    PL_THRESHOLD: "first",
    OPEN: "mean",
    CURRENT: "mean",
}

dashboard = (
    frame.groupby(
        group_columns,
        as_index=False,
        dropna=False,
        sort=False,
        observed=True,
    )
    .agg(aggregations)
)

dashboard[MARKET_AVAILABLE] = (
    dashboard[OPEN].notna()
    & dashboard[CURRENT].notna()
)
dashboard[MARKET_DATA_STATUS] = np.select(
    [
        dashboard[MARKET_AVAILABLE],
        dashboard[OPEN].isna()
        & dashboard[CURRENT].isna(),
        dashboard[OPEN].isna(),
        dashboard[CURRENT].isna(),
    ],
    [
        "Available",
        "Missing Open and Current (Live/OFFICIAL)",
        "Missing Open",
        "Missing Current (Live/OFFICIAL)",
    ],
    default="Incomplete market data",
)
dashboard[MARKET_MOVE] = (
    dashboard[CURRENT] - dashboard[OPEN]
)

columns = [
    SOURCE_TYPE,
    RISK_TYPE,
    RISK_GREEK,
    SPLIT,
    *PORTFOLIO_POSITION_COLUMNS,
    DISPLAY_BUCKET,
    GROUP,
    *([REGION] if REGION in dashboard else []),
    REPORTED_UNDERLYING,
    UNDERLYING,
    *TENOR_COLUMNS,
    *TENOR_ORDER_COLUMNS,
    *PORTFOLIO_REPORTING_COLUMNS,
    PORTFOLIO_MAPPED,
    PROMOTION_REASON,
    PROMOTION_SCORE,
    RISK_THRESHOLD,
    DRISK_THRESHOLD,
    PL_THRESHOLD,
    RISK,
    DRISK,
    OPEN,
    CURRENT,
    PL,
    MARKET_MOVE,
    MARKET_AVAILABLE,
    MARKET_DATA_STATUS,
    *credit_columns,
]

LOGGER.info(
    "Dashboard rows after removing Portfolio: %s -> %s",
    len(frame),
    len(dashboard),
)
return dashboard.loc[:, columns].copy()

```

The financial rules are now explicit:

- Risk, dRisk, P&L, SP01, PSP01, PM01 and other Credit measures: sum.
- Open and Current: mean.

- Move: recalculate from the averaged quotes.
- All-missing additive values: remain missing.
- Rows collapse only when their remaining reporting metadata and market identity agree.

The existing `_validate_dashboard_release()` is already Portfolio-free at its release grain and does not require the raw Portfolio column.

22. Remove Portfolio from the governed UI selector registry

File: `ui/s02_constants.py`

Replace the import

```
from core.s01_schema import PORTFOLIO_COLUMN, PORTFOLIO_FIELDS, PortfolioField
```

with:

```
from core.s01_schema import PORTFOLIO_FIELDS
```

Replace the complete block from `PORTFOLIO_UI_FIELD = ...` through `FILTER_DIMENSION_FIELDS = ...` with

```
VIEW_DIMENSION_FIELDS = tuple(
    field
    for field in PORTFOLIO_FIELDS
    if "view_dimension" in field.roles
)

FILTER_DIMENSION_FIELDS = tuple(
    field
    for field in PORTFOLIO_FIELDS
    if "filter_dimension" in field.roles
)

FILTER_DIMENSION_ORDER = tuple(
    field.key
    for field in FILTER_DIMENSION_FIELDS
)
```

Delete `PORTFOLIO_UI_FIELD` from `__all__` if it is exported later in the file.

After this change:

- View by: Product, Activity, Signoff Group, Category, Sub Category.
- Filters: Activity, Signoff Group, Category, Sub Category.
- Default view: Activity.

23. Remove Portfolio from Quick Search

The Search catalogue currently requires Portfolio. It must be changed in the same patch as `dashboard_frame`, otherwise revision 1 will fail during final catalogue construction.

File: core/s03_search.py

Remove PORTFOLIO_COLUMN from the core.s01_schema import

Delete:

```
PORTFOLIO_COLUMN,
```

Delete:

```
PORTFOLIO = PORTFOLIO_COLUMN
```

Replace PIVOT_INDEX_COLUMNS , GOVERNANCE_COLUMNS , and RISK_ONLY_INDEX_COLUMNS with

```
PIVOT_INDEX_COLUMNS = (
    SOURCE_TYPE,
    RISK_TYPE,
    RISK_GREEK,
    REPORTED_UNDERLYING,
    UNDERLYING,
    *TENOR_COLUMNS,
    *PORTFOLIO_METADATA_COLUMNS,
)
GOVERNANCE_COLUMNS = tuple(PORTFOLIO_METADATA_COLUMNS)
RISK_ONLY_INDEX_COLUMNS = (
    REPORTED_UNDERLYING,
    *GOVERNANCE_COLUMNS,
)
```

Replace QUICK_RISK_FILTER_COLUMNS with

```
QUICK_RISK_FILTER_COLUMNS = (
    SPLIT,
    *(
        field.external_name
        for field in PORTFOLIO_FIELDS
        if "filter_dimension" in field.roles
    ),
)
```

In _risk_pivot_catalog_frame() , replace required with

```
required = [
    SOURCE_TYPE,
    RISK_TYPE,
    RISK_GREEK,
    UNDERLYING,
    *RISK_PIVOT_VALUE_COLUMNS,
]
```

In build_search_catalog() , delete

```
fallback[PORTFOLIO] = UNSPECIFIED
```

File: ui/s04_components.py

In _QUICK_SEARCH_IDENTITY_OPTIONS , delete:

```
("Portfolio", "Portfolio"),
```

Update RISK_FILTER_NOTE to:

```
RISK_FILTER_NOTE = (
    "Include mode uses OR within one populated filter and AND across "
    "populated filters. Exclude mode removes rows matching any selected "
    "value. Leave a filter blank for all values; Risk selections remain "
    "independent from Stock and P&L."
)
```

24. Remove the Portfolio-specific unmapped-books callback dependency

File: `ui/s07_events.py`

Delete the complete `filter_unmapped_portfolios()` function and remove its name from `__all__` .

Then, inside the callback that renders `unmapped-books-grid` , delete this call:

```
frame = filter_unmapped_portfolios(
    frame,
    selected_portfolios,
    exclude_selected=risk_exclude_selected(
        exclude_value
    ),
)
```

Find the callback that renders `unmapped-books-grid` .

Remove these inputs:

```
Input(DIMENSION_FILTER_IDS["portfolio"], "value"),
Input("risk-filter-exclude-selected", "value"),
```

Change the function signature from:

```
def render_unmapped_books(
    _summary_clicks,
    _revision,
    selected_portfolios,
    exclude_value,
    is_open,
):
```

into:

```
def render_unmapped_books(
    _summary_clicks,
    _revision,
    is_open,
):
```

Delete the call to `filter_unmapped_portfolios(...)` .

The unmapped diagnostic table may continue displaying Portfolio because Portfolio is the identity of an unmapped book. It is no longer an analytical page filter.

25. Make P&L history callbacks independent of the number of filters

File: `ui/s08_plevents.py`

Replace the `render_historical_pl_hierarchy()` signature and first lines with

```
def render_historical_pl_hierarchy(
    summary_clicks,
    _row_clicks,
    _period_header_clicks,
    _metric_clicks,
    *args,
):
    """Load and lazily render one expandable Colossus/Predict hierarchy."""
    filter_count = len(PL_FILTER_FIELDS)
    filter_values = args[:filter_count]
    exclude_filter = args[filter_count]
    (
        open_path_tokens,
        open_comparison_tokens,
        selection_state,
    ) = args[filter_count + 1:]
```

Replace the hard-coded `pl_external_filter_map([...])` call with:

```
page_filters = pl_external_filter_map(filter_values)
```

Replace the `render_historical_pl_chart()` signature and first lines with

```
def render_historical_pl_chart(
    selection_state,
    series_choice,
    *args,
):
    """Plot observed Colossus/Predict rows for the selected hierarchy scope."""
    filter_count = len(PL_FILTER_FIELDS)
    filter_values = args[:filter_count]
    exclude_filter = args[filter_count]
    (
        _wtd_clicks,
        _mtd_clicks,
        _ytd_clicks,
        _all_clicks,
        explicit_start,
        explicit_end,
        range_state,
    ) = args[filter_count + 1:]
```

Replace its hard-coded `pl_external_filter_map([...])` call with:

```
pl_external_filter_map(filter_values)
```

File: `ui/s06_plview.py`

Change:

```
"""Build the single authoritative five-field P&L filter row."""
```

into:

```
"""Build the configured P&L filter row."""
```

File: `ui/s14_pl_filters.py`

Change any docstring referring to five selectors into:

```
"""Normalize the configured P&L selectors to external columns."""
```

26. Migrate saved views from five filters to four

The saved-view repository requires exact filter-key equality. Existing JSON files containing `portfolio` will otherwise be rejected.

Create a one-use script outside the application, or run this from the repository root during deployment:

```
import json
from pathlib import Path

for path in Path("data/saved_views").rglob("*.json"):
    payload = json.loads(
        path.read_text(encoding="utf-8")
    )
    payload["filters"].pop("portfolio", None)
    path.write_text(
        json.dumps(payload, indent=2) + "\n",
        encoding="utf-8",
    )
```

No change is needed inside `core/s08_saved_views.py` .

File: `assets/s01_style.css`

Replace:

```
grid-template-columns: repeat(5, minmax(120px, 1fr));
```

with:

```
grid-template-columns: repeat(4, minmax(120px, 1fr));
```

27. Allow partially populated Credit measures

The repository uses the name `PSP01` , not `PSP` .

File: `core/s02_pipeline.py`

Function: `get_product_risk()`

Replace:

```
if credit_measure_columns:
    frame = _coerce_numeric(
        frame,
        credit_measure_columns,
        f"{spec.key} optional Credit measures",
    )
```

with:

```
if credit_measure_columns:
    frame[credit_measure_columns] = frame[
        credit_measure_columns
    ].apply(
        pd.to_numeric,
        errors="raise",
    )
```

Leave mandatory Risk and dRisk coercion unchanged.

File: `ui/s03_aggregate.py`

Function: `credit_measure_available()`

Change:

```
connector_rows[column].notna().all()
```

into:

```
connector_rows[column].notna().any()
```

Function: `credit_measure_values()`

Replace:

```
locally_complete = (
    connector_mask.any()
    and column in frame
    and frame.loc[connector_mask, column].notna().all()
)
use_connector = locally_complete and connector_complete is not False
```

with:

```
locally_available = (
    connector_mask.any()
    and column in frame
    and frame.loc[connector_mask, column].notna().any()
)
use_connector = locally_available and connector_complete is not False
```

Update the docstrings from “complete connector measure” to “available connector measure”.

Do not fill missing Credit values with zero. Existing sums use `min_count=1` .

28. Move the natural tenor sorter into the core schema

File: `core/s01_schema.py`

Add:

```
import re
```

Immediately after `TENOR_ORDER_BY_COLUMN` , add:

```
def tenor_sort_key(value: object) -> tuple[int, float, str]:
    """Sort common tenor labels while retaining unfamiliar valid values."""
    label = str(value).strip()
    upper = label.upper()
    if upper == "SPOT":
        return (-2, 0.0, upper)
    if upper in {"ON", "O/N"}:
        return (-1, 0.0, upper)
    if upper in {"", "N/A", "NA", "UNSPECIFIED"}:
        return (2, float("inf"), upper)
    match = re.search(
        r"(?:^|[\s\-_ /])(\d+(?:\.\d+)?)\s*([DMY])$",
        upper,
    )
    if match:
        number = float(match.group(1))
        days = number * {
            "D": 1.0,
            "M": 30.4375,
            "Y": 365.25,
        }[match.group(2)]
```

```
    return (0, days, upper)
    return (1, float("inf"), upper)
```

Add to `__all__` :

```
"tenor_sort_key",
```

File: `ui/s03_aggregate.py`

Delete:

```
import re
```

Delete the local `tenor_sort_key()` function.

Replace:

```
from core.s01_schema import PORTFOLIO_FIELDS
```

with:

```
from core.s01_schema import PORTFOLIO_FIELDS, tenor_sort_key
```

File: `core/s02_pipeline.py`

Add `tenor_sort_key` to the `core.s01_schema` import.

29. Normalize conflicting or missing tenor orders instead of rejecting them

The policy is:

- 1 Use the lowest supplied rank for each tenor label.
- 2 Sort labels by supplied rank.
- 3 Break collisions with `tenor_sort_key` .
- 4 Put labels with no supplied rank after ranked labels.
- 5 Renumber the final order as `0, 1, 2, ...` .
- 6 Never discard a tenor because its supplied rank conflicts.

File: core/s02_pipeline.py

Replace `_validate_market_tenor_orders()` with

```
def _normalize_market_tenor_orders(
    frame: pd.DataFrame,
    spec: ProductSpec,
) -> pd.DataFrame:
    """Normalize market ranks without dropping any tenor labels."""
    result = frame.copy()

    for axis in spec.axes:
        tenor_column = axis.column
        order_column = axis.order_column

        if order_column not in result:
            result[order_column] = pd.Series(
                pd.NA,
                index=result.index,
                dtype="Int64",
            )

        raw = result[order_column]
        boolean = raw.map(
            lambda value: isinstance(value, (bool, np.bool_))
        )
        numeric = pd.to_numeric(
            raw,
            errors="coerce",
        ).mask(boolean)
        result[order_column] = numeric.astype("Int64")

    groups = result.groupby(
        UNDERLYING,
        sort=False,
        dropna=False,
    ).groups

    for positions in groups.values():
        index = pd.Index(positions)
        authority = (
            result.loc[
                index,
                [tenor_column, order_column],
            ]
            .groupby(
                tenor_column,
                sort=False,
                dropna=False,
            )[order_column]
            .min()
        )

        ordered_tenors = sorted(
            authority.index,
            key=lambda tenor: (
                (
                    float(authority.loc[tenor])
                    if pd.notna(authority.loc[tenor])
                    else float("inf")
                ),
                tenor_sort_key(tenor),
            ),
        )
        rank_by_tenor = {
            tenor: rank
            for rank, tenor in enumerate(ordered_tenors)
        }
        result.loc[index, order_column] = (
            result.loc[index, tenor_column]
            .map(rank_by_tenor)
            .astype("Int64")
        )

    return result
```

Replace all calls to `_validate_market_tenor_orders(...)` with `_normalize_market_tenor_orders(...)` as described in the next steps.

File: ui/s03_aggregate.py

Replace `_resolved_tenor_orders()` with

```
def _resolved_tenor_orders(
    frame: pd.DataFrame,
    *,
    tenor_column: str,
    order_column: str,
) -> pd.Series:
    """Normalize conflicting supplied ranks into one deterministic sequence."""
    if order_column in frame:
        raw = frame[order_column]
        boolean = raw.map(
            lambda value: isinstance(value, (bool, np.bool_))
        )
        supplied = pd.to_numeric(
            raw,
            errors="coerce",
        ).mask(boolean)
    else:
        supplied = pd.Series(
            pd.NA,
            index=frame.index,
            dtype="Float64",
        )

    resolved = pd.Series(
        pd.NA,
        index=frame.index,
        dtype="Int64",
    )
    authority_columns = (
        ["source type", "underlying"]
        if "source type" in frame
        else ["risk type", "risk greek", "underlying"]
    )

    for positions in frame.groupby(
        authority_columns,
        sort=False,
        dropna=False,
    ).groups.values():
        index = pd.Index(positions)
        labels = frame.loc[index, tenor_column].astype(str)
        authority = (
            pd.DataFrame(
                {
                    "label": labels.to_numpy(),
                    "rank": supplied.loc[index].to_numpy(),
                }
            )
            .groupby("label", sort=False)["rank"]
            .min()
        )
        ordered_labels = sorted(
            authority.index,
            key=lambda label: (
                float(authority.loc[label])
                if pd.notna(authority.loc[label])
                else float("inf")
            ),
            tenor_sort_key(label),
        ),
    )
    rank_by_label = {
        label: rank
        for rank, label in enumerate(ordered_labels)
    }
    resolved.loc[index] = (
        labels.map(rank_by_label).astype("Int64")
    )

    return resolved
```

30. Average duplicate market quotes before merging Open and Current

File: `core/s02_pipeline.py`

Add immediately after `_normalize_market_tenor_orders()`

```
def _aggregate_market_leg(
    frame: pd.DataFrame,
    spec: ProductSpec,
    value_column: str,
) -> pd.DataFrame:
    """Average duplicate quotes at one canonical market identity."""
    aggregations: dict[str, object] = {
        value_column: "mean",
    }
    for order_column in spec.tenor_order_columns:
        if order_column in frame:
            aggregations[order_column] = "min"
    if MARKET_STATUS in frame:
        aggregations[MARKET_STATUS] = "first"

    result = (
        frame.groupby(
            list(spec.market_keys),
            as_index=False,
            dropna=False,
            sort=False,
            observed=True,
        )
        .agg(aggregations)
    )

    for order_column in spec.tenor_order_columns:
        if order_column not in result:
            result[order_column] = pd.Series(
                pd.NA,
                index=result.index,
                dtype="Int64",
            )

    return _normalize_market_tenor_orders(result, spec)
```

Replace `get_product_market_open()` with

```
def get_product_market_open(
    spec: ProductSpec,
    open_date: date | datetime | str | pd.Timestamp,
    source: FrameSource,
) -> pd.DataFrame:
    """Load, average, and normalize one product's opening market leg."""
    if source is None:
        raise ProductionIntegrationError(
            f"{spec.key} market open requires a real connector source; provide "
            "source=... or configure ProductConnectorAdapter.market_open"
        )
    _as_timestamp(open_date)
    required = [*spec.market_keys, OPEN]
    columns = [
        *spec.market_keys,
        *spec.tenor_order_columns,
        OPEN,
    ]
    raw_frame = _load_frame(
        source,
        label=f"{spec.key} market open",
        allow_empty=True,
    )
    frame = _enforce_product(
        raw_frame,
        spec,
        f"{spec.key} market open",
    )
    _require_columns(
        frame,
        required,
        f"{spec.key} market open",
    )
    if frame.empty:
        for order_column in spec.tenor_order_columns:
            if order_column not in frame:
                frame[order_column] = pd.Series(
                    pd.NA,
                    index=frame.index,
                    dtype="Int64",
                )
        return frame[columns].copy()

    frame = _require_nonblank(
        frame,
        list(spec.market_keys),
        f"{spec.key} market open",
    )
    frame = _coerce_numeric(
        frame,
        [OPEN],
        f"{spec.key} market open",
    )
    frame = _aggregate_market_leg(frame, spec, OPEN)
    return frame[columns].copy()
```

Replace `get_product_market_status()` with

```
def get_product_market_status(
    spec: ProductSpec,
    market_date: date | datetime | str | pd.Timestamp,
    source: FrameSource,
    *,
    market_status: str,
) -> pd.DataFrame:
    """Load, average, and normalize one product's Current market leg."""
    if source is None:
        raise ProductionIntegrationError(
            f"{spec.key} market status requires a real connector source; provide "
            "source=... or configure ProductConnectorAdapter.market_status"
        )
    _as_timestamp(market_date)
    selected_status = _require_market_status(market_status)
    required = [
        *spec.market_keys,
        CURRENT,
        MARKET_STATUS,
    ]
    columns = [
        *spec.market_keys,
        *spec.tenor_order_columns,
        CURRENT,
        MARKET_STATUS,
    ]

    raw_frame = _load_frame(
        source,
        label=f"{spec.key} market status",
        allow_empty=True,
    )
    frame = _enforce_product(
        raw_frame,
        spec,
        f"{spec.key} market status",
    )
    status_was_supplied = MARKET_STATUS in frame
    if not status_was_supplied:
        frame[MARKET_STATUS] = selected_status

    _require_columns(
        frame,
        required,
        f"{spec.key} current market",
    )
    if frame.empty:
        for order_column in spec.tenor_order_columns:
            if order_column not in frame:
                frame[order_column] = pd.Series(
                    pd.NA,
                    index=frame.index,
                    dtype="Int64",
                )
        return frame[columns].copy()

    frame = _require_nonblank(
        frame,
        list(spec.market_keys),
        f"{spec.key} market status",
    )
    frame = _coerce_numeric(
        frame,
        [CURRENT],
        f"{spec.key} current market",
    )

    if status_was_supplied:
        supplied_status = frame[MARKET_STATUS]
        blank_status = supplied_status.isna() | supplied_status.astype(
            "string"
        ).str.strip().eq("")
        if blank_status.any():
            rows = frame.index[blank_status].tolist()[:5]
            raise ValueError(
                f"{spec.key} market status column {MARKET_STATUS!r} "
                f"has null or blank values at rows {rows}"
            )
        exact_status = supplied_status.map(
            lambda value: (
                isinstance(value, str)
                and value == selected_status
            )
        )
```

```

    )
    if not exact_status.all():
        raise ValueError(
            f"{spec.key} market status must be exactly "
            f"{selected_status!r} on every supplied row"
        )
    else:
        frame[MARKET_STATUS] = selected_status

    frame = _aggregate_market_leg(
        frame,
        spec,
        CURRENT,
    )
    return frame[columns].copy()

```

Replace `_merge_validated_market_legs()` with

```

def _merge_validated_market_legs(
    spec: ProductSpec,
    market_open: pd.DataFrame,
    market_status: pd.DataFrame,
    *,
    selected_status: str,
) -> pd.DataFrame:
    """Outer-merge averaged quote legs and derive one final tenor order."""
    selected_status = _require_market_status(selected_status)

    market = market_open.drop(
        columns=list(spec.tenor_order_columns),
        errors="ignore",
    ).merge(
        market_status.drop(
            columns=list(spec.tenor_order_columns),
            errors="ignore",
        ),
        on=spec.market_keys,
        how="outer",
        validate="one_to_one",
        indicator=True,
    )
    market = _normalize_market_tenor_orders(
        market,
        spec,
    )
    market[MARKET_STATUS] = selected_status
    market[MARKET_AVAILABLE] = (
        market["_merge"].eq("both")
        & market[OPEN].notna()
        & market[CURRENT].notna()
    )
    market[MARKET_DATA_STATUS] = np.select(
        [
            market[MARKET_AVAILABLE],
            market[OPEN].isna() & market[CURRENT].isna(),
            market[OPEN].isna(),
            market[CURRENT].isna(),
        ],
        [
            "Available",
            "Missing Open and Current (Live/OFFICIAL)",
            "Missing Open",
            "Missing Current (Live/OFFICIAL)",
        ],
        default="Incomplete market data",
    )
    market = market.drop(columns="_merge")
    market[MARKET_MOVE] = market[CURRENT] - market[OPEN]
    return market

```

This removes all three strict failure modes:

- duplicate quote keys;
- conflicting tenor ranks;
- Open/Current rank disagreement.

31. Display Open, Current, and Move at Reported Underlying

The quote hierarchy already deduplicates repeated quote identities and calculates equal-weight market values. The display rule currently hides those values at Reported Underlying.

File: `ui/s03_aggregate.py`

Function: `should_show_sum()`

Replace:

```
if column in {"move", "open", "current"}:  
    return _is_semantic_underlying(context)
```

with:

```
if column in {"move", "open", "current"}:  
    return (  
        "reported underlying" in context  
        or _is_semantic_underlying(context)  
    )
```

Do not rewrite `hierarchical_market_value()` or `_MarketQuoteIndex` .

32. Replace the custom Stock hierarchy with a P&L-style Aggregate Stock page

The existing Stock page maintains a custom hierarchy, promotion threshold, temporary currency grouping, JSON path tokens, and open-row state. That machinery is not required for the requested page.

The replacement keeps:

- current and prior date controls;
- saved views;
- the four governed reporting filters;
- mapped/unmapped counters;
- lazy detailed comparison rows.

It replaces the hierarchy with:

- an always-visible **Aggregate Stock** section;
- the same governed **View by** choices as Aggregate P&L;
- Prior Quantity, Current Quantity, Quantity Change;
- Prior Market Value, Current Market Value, Market Value Change.

Portfolio remains in the raw Stock connector and mapping join, but it is removed before the public Stock page receives the mapped comparison.

32.1 Aggregate duplicate raw Stock rows instead of rejecting them

File: `core/s07_stock.py`

Add immediately before `compare_stock_snapshots()`

```
def _aggregate_stock_snapshot(
    frame: pd.DataFrame,
) -> pd.DataFrame:
    """Sum duplicate source rows at the current Stock identity."""
    return (
        frame.groupby(
            list(STOCK_IDENTITY_COLUMNS),
            as_index=False,
            dropna=False,
            sort=False,
            observed=True,
        )[list(STOCK_NUMERIC_COLUMNS)]
        .sum(min_count=1)
    )
```

Replace the first four lines inside `compare_stock_snapshots()`

Replace:

```
current = validate_stock_frame(current_stock, label="Current Stock")
prior = validate_stock_frame(prior_stock, label="Prior Stock")
_reject_duplicate_stock_identity(current, label="Current Stock")
_reject_duplicate_stock_identity(prior, label="Prior Stock")
```

with:

```
current = _aggregate_stock_snapshot(
    validate_stock_frame(
        current_stock,
        label="Current Stock",
    )
)
prior = _aggregate_stock_snapshot(
    validate_stock_frame(
        prior_stock,
        label="Prior Stock",
    )
)
```

Delete the complete `_reject_duplicate_stock_identity()` function.

Update the `compare_stock_snapshots()` docstring to:

```
"""Outer-compare two Stock snapshots after summing duplicate source rows."""
```

32.2 Remove Portfolio after the Stock mapping join

File: `core/s07_stock.py`

Replace `MAPPED_STOCK_COMPARISON_COLUMNS`

```
MAPPED_STOCK_COMPARISON_COLUMNS = (
    "CRDS",
    "CPTY",
    "Instrument",
    "Currency",
    *STOCK_COMPARISON_NUMERIC_COLUMNS,
    STOCK_CHANGE_COLUMN,
    *PORTFOLIO_METADATA_COLUMNS,
    PORTFOLIO_MAPPED_COLUMN,
)
```

Replace STOCK_FILTER_COLUMN_BY_KEY

```
STOCK_FILTER_COLUMN_BY_KEY = {  
    field.key: field.external_name  
    for field in PORTFOLIO_FIELDS  
    if "filter_dimension" in field.roles  
}
```

Add immediately before `map_stock_comparison_portfolios()`

```
def _collapse_mapped_stock_portfolios(
    mapped: pd.DataFrame,
) -> pd.DataFrame:
    """Remove Portfolio after mapping and recompute comparison status."""
    group_columns = [
        "CRDS",
        "CPTY",
        "Instrument",
        "Currency",
        *PORTFOLIO_METADATA_COLUMNS,
        PORTFOLIO_MAPPED_COLUMN,
    ]
    value_columns = [
        PRIOR_QUANTITY_COLUMN,
        CURRENT_QUANTITY_COLUMN,
        PRIOR_MARKET_VALUE_COLUMN,
        CURRENT_MARKET_VALUE_COLUMN,
    ]

    result = (
        mapped.groupby(
            group_columns,
            as_index=False,
            dropna=False,
            sort=False,
            observed=True,
        )[value_columns]
        .sum(min_count=1)
    )

    result[QUANTITY_CHANGE_COLUMN] = (
        result[CURRENT_QUANTITY_COLUMN].fillna(0.0)
        - result[PRIOR_QUANTITY_COLUMN].fillna(0.0)
    )
    result[MARKET_VALUE_CHANGE_COLUMN] = (
        result[CURRENT_MARKET_VALUE_COLUMN].fillna(0.0)
        - result[PRIOR_MARKET_VALUE_COLUMN].fillna(0.0)
    )

    prior_present = result[
        [
            PRIOR_QUANTITY_COLUMN,
            PRIOR_MARKET_VALUE_COLUMN,
        ]
    ].notna().any(axis=1)
    current_present = result[
        [
            CURRENT_QUANTITY_COLUMN,
            CURRENT_MARKET_VALUE_COLUMN,
        ]
    ].notna().any(axis=1)
    unchanged = (
        prior_present
        & current_present
        & result[CURRENT_QUANTITY_COLUMN]
        .fillna(0.0)
        .eq(result[PRIOR_QUANTITY_COLUMN].fillna(0.0))
        & result[CURRENT_MARKET_VALUE_COLUMN]
        .fillna(0.0)
        .eq(result[PRIOR_MARKET_VALUE_COLUMN].fillna(0.0))
    )

    result[STOCK_CHANGE_COLUMN] = np.select(
        [
            current_present & ~prior_present,
            prior_present & ~current_present,
            unchanged,
        ],
        [
            "Added",
            "Removed",
            "Unchanged",
        ],
        default="Changed",
    )
    return result.loc[
        :,
        list(MAPPED_STOCK_COMPARISON_COLUMNS),
    ].copy()
```

Replace `map_stock_comparison_portfolios()` completely

```
def map_stock_comparison_portfolios(
    current_stock: pd.DataFrame,
    prior_stock: pd.DataFrame,
    portfolio_config: pd.DataFrame | str | Path,
) -> pd.DataFrame:
    """Map through Portfolio, then publish a Portfolio-free comparison."""
    comparison = compare_stock_snapshots(
        current_stock,
        prior_stock,
    )
    mapped = merge_config(
        comparison,
        portfolio_config,
    )
    return _collapse_mapped_stock_portfolios(mapped)
```

Do not change `STOCK_TEXT_COLUMNS` , `STOCK_COLUMNS` , or `STOCK_IDENTITY_COLUMNS` . The raw connector still needs Portfolio to identify and map source rows.

32.3 Load current Stock, prior Stock, and Portfolio mapping concurrently

This is a separate task pool inside the one Gunicorn process. It is not an additional Gunicorn worker.

Use one worker for the checked-in fake fixture. Use three workers only after the real Stock and Portfolio clients are confirmed thread-safe and have native I/O timeouts.

File: `ui/s10_stock.py`

Add imports near the top

```
import os
from concurrent.futures import (
    ALL_COMPLETED,
    ThreadPoolExecutor,
    wait,
)
```

Keep `json` only until the old hierarchy code is deleted later in this section.

Add above `load_stock_page_data()`

```
def _positive_stock_setting(
    name: str,
    default: int,
) -> int:
    raw = os.getenv(name, str(default))
    try:
        value = int(raw)
    except (TypeError, ValueError) as exc:
        raise ValueError(
            f"{name} must be a positive integer"
        ) from exc
    if value < 1:
        raise ValueError(
            f"{name} must be a positive integer"
        )
    return value

def _positive_stock_timeout(
    name: str,
    default: float,
) -> float:
    raw = os.getenv(name, str(default))
    try:
        value = float(raw)
    except (TypeError, ValueError) as exc:
        raise ValueError(
            f"{name} must be a positive number"
        ) from exc
    if value <= 0:
        raise ValueError(
            f"{name} must be greater than zero"
        )
    return value
```

Replace `load_stock_page_data()` completely

```
def load_stock_page_data(
    *,
    stock_source: StockSource | StockConnectorAdapter,
    portfolio_config_source: (
        pd.DataFrame
        | str
        | Path
        | Callable[
            [pd.Timestamp],
            pd.DataFrame | str | Path,
        ]
    ),
    current_date: object,
    prior_date: object,
    portfolio_date: object | None = None,
) -> StockPageData:
    """Load two dated Stock legs and mapping with bounded source concurrency."""
    current, prior = normalize_stock_date_pair(
        current_date,
        prior_date,
    )
    selected_portfolio_date = normalize_stock_date(
        current
        if portfolio_date is None
        else portfolio_date
    )
    adapter = (
        stock_source
        if isinstance(stock_source, StockConnectorAdapter)
        else build_stock_adapter(stock=stock_source)
    )

    def load_portfolio_config():
        return (
            portfolio_config_source(
                selected_portfolio_date
            )
            if callable(portfolio_config_source)
            else portfolio_config_source
        )

    workers = min(
        3,
        _positive_stock_setting(
            "CUBE_STOCK_CONNECTOR_WORKERS",
            1,
        ),
    )

    if workers == 1:
        current_stock = adapter.get_stock(current)
        prior_stock = adapter.get_stock(prior)
        portfolio_config = load_portfolio_config()
    else:
        timeout = _positive_stock_timeout(
            "CUBE_STOCK_BATCH_TIMEOUT_SECONDS",
            180.0,
        )
        executor = ThreadPoolExecutor(
            max_workers=workers,
            thread_name_prefix="cube-stock-source",
        )
        futures = {
            "current": executor.submit(
                adapter.get_stock,
                current,
            ),
            "prior": executor.submit(
                adapter.get_stock,
                prior,
            ),
            "portfolio": executor.submit(
                load_portfolio_config,
            ),
        }
        done, pending = wait(
            tuple(futures.values()),
            timeout=timeout,
            return_when=ALL_COMPLETED,
        )
        if pending:
            pending_names = [
                name
```

```

        for name, future in futures.items()
        if future in pending
    ]
    for future in pending:
        future.cancel()
    executor.shutdown(
        wait=False,
        cancel_futures=True,
    )
    raise TimeoutError(
        "Stock source batch exceeded "
        f"{timeout:g}s; pending={pending_names}"
    )
try:
    current_stock = futures["current"].result()
    prior_stock = futures["prior"].result()
    portfolio_config = futures[
        "portfolio"
    ].result()
finally:
    executor.shutdown(wait=True)

mapped = map_stock_comparison_portfolios(
    current_stock,
    prior_stock,
    portfolio_config,
)
return StockPageData(
    mapped_stock=mapped,
    current_date=current,
    prior_date=prior,
    portfolio_date=selected_portfolio_date,
)

```

A framework timeout cannot kill a running synchronous library call. Configure finite timeouts in the real Stock and Portfolio clients as well.

32.4 Add the P&L-style Aggregate Stock table

File: `ui/s10_stock.py`

Replace the `.s02_constants` import with

```

from .s02_constants import (
    DEFAULT_VIEW_DIMENSION,
    FILTER_DIMENSION_FIELDS,
    VIEW_DIMENSION_FIELDS,
)

```

Add below `STOCK_FILTER_NOTE`

```

STOCK_VIEW_COLUMN_BY_KEY = {
    field.key: field.external_name
    for field in VIEW_DIMENSION_FIELDS
}

```

Add immediately above `build_stock_page_from_data()`

```
def build_stock_aggregate_table(
    mapped_stock: pd.DataFrame,
    dimension: str | None,
) -> object:
    """Aggregate the filtered Stock comparison by one governed dimension."""
    selected_dimension = (
        dimension
        if dimension in STOCK_VIEW_COLUMN_BY_KEY
        else DEFAULT_VIEW_DIMENSION
    )
    group_column = STOCK_VIEW_COLUMN_BY_KEY[
        selected_dimension
    ]

    if mapped_stock.empty:
        return html.Div(
            "No Stock rows match the selected filters.",
            className="static-data-empty",
        )

    source_columns = [
        PRIOR_QUANTITY_COLUMN,
        CURRENT_QUANTITY_COLUMN,
        PRIOR_MARKET_VALUE_COLUMN,
        CURRENT_MARKET_VALUE_COLUMN,
    ]
    frame = (
        mapped_stock.groupby(
            group_column,
            as_index=False,
            dropna=False,
            sort=False,
            observed=True,
        )[source_columns]
        .sum(min_count=1)
    )
    frame[QUANTITY_CHANGE_COLUMN] = (
        frame[CURRENT_QUANTITY_COLUMN].fillna(0.0)
        - frame[PRIOR_QUANTITY_COLUMN].fillna(0.0)
    )
    frame[MARKET_VALUE_CHANGE_COLUMN] = (
        frame[CURRENT_MARKET_VALUE_COLUMN].fillna(0.0)
        - frame[PRIOR_MARKET_VALUE_COLUMN].fillna(0.0)
    )
    frame["__sort__"] = frame[
        CURRENT_MARKET_VALUE_COLUMN
    ].abs().fillna(0.0)
    frame = (
        frame.sort_values(
            ["__sort__", group_column],
            ascending=[False, True],
            kind="stable",
        )
        .drop(columns="__sort__")
    )

    display_columns = [
        group_column,
        PRIOR_QUANTITY_COLUMN,
        CURRENT_QUANTITY_COLUMN,
        QUANTITY_CHANGE_COLUMN,
        PRIOR_MARKET_VALUE_COLUMN,
        CURRENT_MARKET_VALUE_COLUMN,
        MARKET_VALUE_CHANGE_COLUMN,
    ]
    numeric_columns = set(display_columns[1:])

    return dash_table.DataTable(
        id="stock-aggregate-table",
        columns=[
            {
                "name": column,
                "id": column,
                **({
                    {
                        "type": "numeric",
                        "format": {"specifier": ",.2f"},
                    }
                    if column in numeric_columns
                    else {}
                },
            ),
            for column in display_columns
        ],
    )
```

```

],
data=_json_records(frame[display_columns]),
editable=False,
filter_action="native",
filter_options={"case": "insensitive"},
sort_action="native",
sort_mode="multi",
page_action="native",
page_size=50,
fixed_rows={"headers": True},
style_table={
    "overflowX": "auto",
    "maxHeight": "72vh",
},
style_header={
    "backgroundColor": "#E3E5E7",
    "color": "#111111",
    "fontWeight": "700",
    "border": "1px solid #D9E0E7",
},
style_cell={
    "backgroundColor": "#FFFFFF",
    "color": "#111111",
    "border": "1px solid #E5E9ED",
    "fontFamily": (
        "Inter, Segoe UI, Arial, sans-serif"
    ),
    "fontSize": "12px",
    "padding": "8px 10px",
    "textAlign": "left",
    "minWidth": "110px",
    "whiteSpace": "nowrap",
},
style_cell_conditional=[
    {
        "if": {
            "column_id": list(numeric_columns)
        },
        "fontVariantNumeric": "tabular-nums",
        "textAlign": "right",
    }
],
style_data_conditional=[
    {
        "if": {
            "filter_query": (
                f"{{{MARKET_VALUE_CHANGE_COLUMN}}} < 0"
            ),
            "column_id": (
                MARKET_VALUE_CHANGE_COLUMN
            ),
        },
        "color": "#B42318",
    }
],
)

def _stock_aggregate_section(
    initial_frame: pd.DataFrame | None = None,
) -> html.Section:
    """Build the always-visible Aggregate Stock section."""
    options = [
        {
            "label": field.label,
            "value": field.key,
        }
        for field in VIEW_DIMENSION_FIELDS
    ]
    initial = (
        build_stock_aggregate_table(
            initial_frame,
            DEFAULT_VIEW_DIMENSION,
        )
        if initial_frame is not None
        else html.Div(
            "Stock data is still loading.",
            className="empty-state",
            role="status",
        )
    )
    return html.Section(
        [
            html.H2(
                "Aggregate Stock",
                className=(

```

```

        "aux-summary aggregate-pl-summary "
        "pnl-static-heading"
    ),
),
html.Div(
    [
        html.Div(
            "View by",
            className="aggregate-pl-title",
        ),
        dcc.RadioItems(
            id="stock-aggregate-dimension",
            options=options,
            value=DEFAULT_VIEW_DIMENSION,
            inline=True,
            className="aggregate-pl-selector",
        ),
    ],
    className="aggregate-pl-header",
),
html.Div(
    html.Div(
        initial,
        id="stock-aggregate-grid",
    ),
    className="aggregate-pl-panel",
),
],
className=(
    "aux-details aggregate-pl-details "
    "pnl-always-open-section"
),
)

```

32.5 Replace the loaded Stock page body

File: `ui/s10_stock.py`

Replace `build_stock_page_from_data()` completely

```
def build_stock_page_from_data(
    page_data: StockPageData,
    *,
    selected_filters: (
        Mapping[str, Sequence[str] | None]
        | None
    ) = None,
    exclude_selected: bool = False,
) -> html.Main:
    """Build the simple Aggregate Stock and lazy-detail page."""
    filtered = filter_stock_comparison(
        page_data.mapped_stock,
        dict(selected_filters or {}),
        exclude_selected=exclude_selected,
    )
    rows, mapped, unmapped = stock_summary_text(
        filtered,
        total_rows=len(page_data.mapped_stock),
        current_date=page_data.current_date,
        prior_date=page_data.prior_date,
    )
    return html.Main(
        [
            html.Div(
                [
                    html.Span(
                        rows,
                        id="stock-row-count",
                        className="static-data-row-count",
                    ),
                    html.Span(
                        mapped,
                        id="stock-mapped-count",
                        className="static-data-col-count",
                    ),
                    html.Span(
                        unmapped,
                        id="stock-unmapped-count",
                        className="static-data-col-count",
                    ),
                ],
                className="static-data-meta",
            ),
            _stock_aggregate_section(filtered),
            build_stock_source_rows_section(),
        ],
        id="stock-comparison-view",
        **{
            "data-stock-columns": ", ".join(
                STOCK_COLUMNS
            ),
            "data-current-date": (
                page_data.current_date.date().isoformat()
            ),
            "data-prior-date": (
                page_data.prior_date.date().isoformat()
            ),
        },
    )
```

Delete the obsolete `promotion_threshold` parameter from callers of this function.

32.6 Replace the Stock placeholder

Replace `build_stock_page_placeholder()` completely

```
def build_stock_page_placeholder(
    message: str,
    *,
    error: bool = False,
) -> list[object]:
    """Keep every Stock callback target mounted before data is available."""
    status = (
        str(message).strip()
        or "Stock data is not available yet."
    )
    return [
        (
            html.P(
                status,
                id="stock-load-error",
                className="static-data-empty",
                role="alert",
            )
            if error
            else None
        ),
        html.Div(
            [
                html.Span(
                    "Rows: loading...",
                    id="stock-row-count",
                    className="static-data-row-count",
                ),
                html.Span(
                    "Mapped: loading...",
                    id="stock-mapped-count",
                    className="static-data-col-count",
                ),
                html.Span(
                    "Unmapped: loading...",
                    id="stock-unmapped-count",
                    className="static-data-col-count",
                ),
            ],
            className="static-data-meta",
        ),
        _stock_aggregate_section(None),
        build_stock_source_rows_section(status),
    ]
```

32.7 Replace the complete Stock shell

Replace `build_stock_page_shell()` completely

```
def build_stock_page_shell(
    *,
    current_date: object,
    prior_date: object,
) -> html.Main:
    """Paint the complete Stock control shell before connector work."""
    current, prior = normalize_stock_date_pair(
        current_date,
        prior_date,
    )
    filter_controls = [
        html.Div(
            [
                html.Label(
                    field.label,
                    htmlFor=STOCK_FILTER_IDS[field.key],
                ),
                dcc.Dropdown(
                    id=STOCK_FILTER_IDS[field.key],
                    options=[],
                    multi=True,
                    placeholder=(
                        f"All {field.label.casefold()} values"
                    ),
                    value=[],
                ),
            ],
            className="control-field",
        )
        for field in STOCK_FILTER_FIELDS
    ]

    return html.Main(
        [
            dcc.Store(
                id="stock-loaded-revision",
                data=-1,
            ),
            dcc.Store(
                id="stock-loaded-dates",
                data=None,
            ),
            dcc.Store(
                id="stock-source-rows-state",
                data={
                    "requested": False,
                    "loaded_dates": None,
                },
            ),
            dcc.Store(
                id="stock-dimension-filter-store",
                data={
                    "filters": {
                        field.key: []
                        for field in STOCK_FILTER_FIELDS
                    },
                    "exclude_selected": False,
                },
            ),
            dcc.Interval(
                id="stock-load-trigger",
                interval=1_000,
                n_intervals=0,
                disabled=False,
            ),
            html.Div(
                [
                    html.H2(
                        "Stock",
                        className="static-data-page-title",
                    ),
                    html.P(
                        (
                            "Compare current and prior Stock "
                            "using governed reporting dimensions."
                        ),
                        className="static-data-page-note",
                    ),
                ],
                className="static-data-header",
            ),
        ],
    )
```

```

    ),
    html.Div(
        [
            html.Div(
                [
                    html.Label(
                        "Current stock date",
                        htmlFor="stock-current-date",
                    ),
                    dcc.DatePickerSingle(
                        id="stock-current-date",
                        date=(
                            current.date().isoformat()
                        ),
                        display_format="YYYY-MM-DD",
                        clearable=False,
                    ),
                ],
                className="control-field",
            ),
            html.Div(
                [
                    html.Label(
                        "Prior stock date",
                        htmlFor="stock-prior-date",
                    ),
                    dcc.DatePickerSingle(
                        id="stock-prior-date",
                        date=prior.date().isoformat(),
                        display_format="YYYY-MM-DD",
                        clearable=False,
                    ),
                ],
                className="control-field",
            ),
            html.Div(
                [
                    html.Label(
                        "Compare",
                        htmlFor="stock-compare-button",
                    ),
                    html.Button(
                        "Compare dates",
                        id="stock-compare-button",
                        n_clicks=0,
                        type="button",
                        className=(
                            "refresh-button "
                            "stock-compare-button"
                        ),
                    ),
                ],
                className=(
                    "control-field "
                    "stock-compare-action"
                ),
            ),
        ],
        className="controls top-controls",
    ),
    build_saved_filter_view_bar(
        STOCK_SAVED_VIEW_CONTROLS,
        filter_note=STOCK_FILTER_NOTE,
        filter_bar=html.Div(
            [
                html.Div(
                    [
                        *filter_controls,
                        dcc.Checklist(
                            id=(
                                "stock-filter-"
                                "exclude-selected"
                            ),
                            options=[
                                {
                                    "label": (
                                        "Exclude rows matching "
                                        "any selected value"
                                    ),
                                    "value": "exclude",
                                }
                            ],
                            value=[],
                            className=(
                                "stock-filter-mode "
                                "filter-mode-control"
                            )
                        )
                    ]
                )
            ]
        )
    )

```

```

        ),
        ],
        className=(
            "controls filter-controls"
        ),
    ),
    ],
    className=(
        "dimension-filter-bar top-controls"
    ),
),
),
dcc.Loading(
    html.Div(
        build_stock_page_placeholder(
            (
                "Loading current and prior Stock "
                "and the Portfolio mapping..."
            )
        ),
        id="stock-page-content",
    ),
    delay_show=120,
),
],
id="stock-page",
className="static-data-page",
)

```

32.8 Replace the Stock filtering callback

File: ui/s09_factory.py

Replace the Stock imports from `.s10_stock`

Remove:

```
STOCK_HIERARCHY_TOGGLE_TYPE
build_stock_hierarchy_panel_with_state
normalize_stock_promotion_threshold
normalize_stock_hierarchy_open_tokens
toggle_stock_hierarchy_open_tokens
```

Add:

```
build_stock_aggregate_table
```

Replace the complete current `filter_stock_table()` callback decorator and function

```
@app.callback(
    Output("stock-row-count", "children"),
    Output("stock-mapped-count", "children"),
    Output("stock-unmapped-count", "children"),
    Output(
        "stock-dimension-filter-store",
        "data",
    ),
    Output("stock-aggregate-grid", "children"),
    *[
        Input(
            STOCK_FILTER_IDS[field.key],
            "value",
        )
        for field in STOCK_FILTER_FIELDS
    ],
    Input(
        "stock-filter-exclude-selected",
        "value",
    ),
    Input(
        "stock-aggregate-dimension",
        "value",
    ),
    Input("stock-loaded-dates", "data"),
    prevent_initial_call=True,
)
def filter_stock_table(*values):
    """Filter the cached Stock comparison and rebuild Aggregate Stock."""
    filter_count = len(STOCK_FILTER_FIELDS)
    selected_filter_values = values[:filter_count]
    exclude_value = values[filter_count]
    dimension = values[filter_count + 1]
    loaded_dates = values[filter_count + 2]

    key = stock_cache_key(loaded_dates)
    page_data = stock_cached_pages.get(key)
    if page_data is None:
        return (no_update,) * 5

    selected_filters = stock_filter_map(
        selected_filter_values
    )
    exclude_selected = stock_exclude_selected(
        exclude_value
    )
    filtered = filter_stock_comparison(
        page_data.mapped_stock,
        selected_filters,
        exclude_selected=exclude_selected,
    )
    rows, mapped, unmapped = stock_summary_text(
        filtered,
        total_rows=len(page_data.mapped_stock),
        current_date=page_data.current_date,
        prior_date=page_data.prior_date,
    )
    return (
        rows,
        mapped,
        unmapped,
        {
            "filters": selected_filters,
            "exclude_selected": exclude_selected,
        },
        build_stock_aggregate_table(
            filtered,
            dimension,
        ),
    )
```

Keep the lazy `render_stock_source_rows()` callback.

32.9 Reduce the process-local Stock cache

File: `ui/s09_factory.py`

In `load_stock_revision()` , replace:

```
if len(stock_cached_pages) > 8:
```

with:

```
if len(stock_cached_pages) > 2:
```

32.10 Update `build_stock_table()` for the Portfolio-free mapped frame

File: `ui/s10_stock.py`

The existing function can keep its DataTable styling, but remove this tooltip entry:

```
"Portfolio": "Stock Portfolio used for the governed mapping",
```

The table automatically follows the new `MAPPED_STOCK_COMPARISON_COLUMNS` and therefore no longer exposes Portfolio.

Rename the source-row disclosure labels because the mapped comparison is now Portfolio-collapsed:

Replace:

```
html.Summary("Source comparison rows")
```

with:

```
html.Summary("Detailed comparison rows")
```

Replace:

```
"Source comparison rows are not loaded. Load them only when needed."
```

with:

```
"Detailed comparison rows are not loaded. Load them only when needed."
```

Replace button text:

```
"Load filtered source rows"
```

with:

```
"Load filtered detail rows"
```

Update the corresponding strings returned by `render_stock_source_rows()` in `ui/s09_factory.py` .

32.11 Delete the obsolete Stock hierarchy code

After the new page passes its focused tests, delete from `core/s07_stock.py` :

```
STOCK_PROMOTION_BUCKET_COLUMN
STOCK_TEMPORARY_GROUP_COLUMN
STOCK_PROMOTION_THRESHOLD_DEFAULT
STOCK_HIERARCHY_COLUMNS
STOCK_PROMOTION_IDENTITY_COLUMNS
STOCK_HIERARCHY_* constants
normalize_stock_promotion_threshold()
prepare_stock_hierarchy()
_stock_hierarchy_metrics()
_ordered_stock_hierarchy_children()
summarize_stock_hierarchy()
```

```
summarize_visible_stock_hierarchy()
```

Delete from `ui/s10_stock.py` :

```
STOCK_HIERARCHY_METRICS
STOCK_HIERARCHY_TOGGLE_TYPE
stock_hierarchy_path_token()
stock_hierarchy_path_from_token()
normalize_stock_hierarchy_open_tokens()
toggle_stock_hierarchy_open_tokens()
all hierarchy row/cell builders
build_stock_hierarchy* functions
```

Then remove unused imports:

```
json
ROW_TOGGLE_CLOSED_GLYPH
ROW_TOGGLE_OPEN_GLYPH
all removed hierarchy constants/functions
```

Remove the deleted names from both modules' `__all__` lists.

`pages/stock.py` requires no change.

32.12 Replace the pure Stock page helpers and final exports

File: `ui/s10_stock.py`

Replace `build_stock_page()` completely

```
def build_stock_page(
    current_stock: pd.DataFrame,
    prior_stock: pd.DataFrame,
    portfolio_config: pd.DataFrame | str | Path,
    *,
    current_date: object,
    prior_date: object,
    selected_filters: (
        Mapping[str, Sequence[str] | None]
        | None
    ) = None,
    exclude_selected: bool = False,
) -> html.Main:
    """Build the pure Portfolio-free Aggregate Stock page."""
    current, prior = normalize_stock_date_pair(
        current_date,
        prior_date,
    )
    data = StockPageData(
        mapped_stock=map_stock_comparison_portfolios(
            current_stock,
            prior_stock,
            portfolio_config,
        ),
        current_date=current,
        prior_date=prior,
        portfolio_date=current,
    )
    return build_stock_page_from_data(
        data,
        selected_filters=selected_filters,
        exclude_selected=exclude_selected,
    )
```

Replace `build_stock_page_from_sources()` completely

```
def build_stock_page_from_sources(
    *,
    stock_source: StockSource | StockConnectorAdapter,
    portfolio_config_source: (
        pd.DataFrame
        | str
        | Path
        | Callable[
            [pd.Timestamp],
            pd.DataFrame | str | Path,
        ]
    ),
    current_date: object,
    prior_date: object,
    portfolio_date: object | None = None,
    selected_filters: (
        Mapping[str, Sequence[str] | None]
        | None
    ) = None,
    exclude_selected: bool = False,
) -> html.Main:
    """Load both Stock legs, then build Aggregate Stock."""
    page_data = load_stock_page_data(
        stock_source=stock_source,
        portfolio_config_source=(
            portfolio_config_source
        ),
        current_date=current_date,
        prior_date=prior_date,
        portfolio_date=portfolio_date,
    )
    return build_stock_page_from_data(
        page_data,
        selected_filters=selected_filters,
        exclude_selected=exclude_selected,
    )
```

Replace the complete `ui/s10_stock.py::__all__` list with

```
__all__ = [
    "STOCK_FILTER_FIELDS",
    "STOCK_FILTER_IDS",
    "STOCK_SAVED_VIEW_CONTROLS",
    "StockPageData",
    "build_stock_aggregate_table",
    "build_stock_page",
    "build_stock_page_from_data",
    "build_stock_page_from_sources",
    "build_stock_page_placeholder",
    "build_stock_page_shell",
    "build_stock_source_rows_section",
    "build_stock_table",
    "build_stock_table_panel",
    "default_stock_dates",
    "load_stock_page_data",
    "normalize_stock_date_pair",
    "stock_exclude_selected",
    "stock_filter_map",
    "stock_filter_options",
    "stock_summary_text",
]
```

File: `core/s07_stock.py`

Replace the complete `core/s07_stock.py: :__all__` list with

```
__all__ = [  
    "CURRENT_MARKET_VALUE_COLUMN",  
    "CURRENT_QUANTITY_COLUMN",  
    "MARKET_VALUE_CHANGE_COLUMN",  
    "MAPPED_STOCK_COLUMNS",  
    "MAPPED_STOCK_COMPARISON_COLUMNS",  
    "PRIOR_MARKET_VALUE_COLUMN",  
    "PRIOR_QUANTITY_COLUMN",  
    "QUANTITY_CHANGE_COLUMN",  
    "STOCK_CHANGE_COLUMN",  
    "STOCK_COLUMNS",  
    "STOCK_COMPARISON_COLUMNS",  
    "STOCK_COMPARISON_NUMERIC_COLUMNS",  
    "STOCK_FILTER_COLUMN_BY_KEY",  
    "STOCK_IDENTITY_COLUMNS",  
    "STOCK_NUMERIC_COLUMNS",  
    "STOCK_TEXT_COLUMNS",  
    "compare_stock_snapshots",  
    "filter_stock_comparison",  
    "map_stock_comparison_portfolios",  
    "map_stock_portfolios",  
    "validate_stock_frame",  
]
```

33. Keep the fake connector simple and deterministic

The fake connector is already more efficient than the recovered recommendation suggests:

- `_load_fake_csv()` is cached by file path, modification time and size.
- `_load_fake_source_partition()` is cached by dataset, file revision, Source Type and Underlying.
- callers receive defensive narrow copies.

Do not create a separate `asyncio` implementation for `pandas.read_csv()`. `read_csv()` is blocking, and the fixture data is already parsed once per file revision.

33.1 Keep fixture worker counts at one

For local tests and the checked-in fake data:

```
export CUBE_RISK_CONNECTOR_WORKERS=1  
export CUBE_MARKET_CONNECTOR_WORKERS=1  
export CUBE_STOCK_CONNECTOR_WORKERS=1
```

The manager-level concurrency code remains exercised by dedicated sleeping test connectors. It does not need to make the tiny fixture slower during every test.

33.2 Optional: reduce repeated `stat()` calls on network storage

Use this only when the fixture files are on NFS or another slow metadata filesystem.

File: `feeds/s01_sources.py`

Add this import near the top:

```
import time
```

Then add:

```
_FAKE_REVISION_TTL_SECONDS = max(  
    0.0,
```

```

float(
    os.getenv(
        "CUBE_FAKE_REVISION_TTL_SECONDS",
        "0",
    )
),
)

@lru_cache(maxsize=64)
def _cached_fake_csv_revision(
    dataset: str,
    ttl_bucket: int,
) -> tuple[Path, str, int, int]:
    """Read one file revision per short metadata TTL bucket."""
    del ttl_bucket

    path = FAKE_CSV_FILES[dataset]
    stat = path.stat()
    return (
        path,
        str(path.resolve()),
        stat.st_mtime_ns,
        stat.st_size,
    )

```

Replace `_fake_csv_revision()` with:

```

def _fake_csv_revision(
    dataset: str,
) -> tuple[Path, str, int, int]:
    """Return one file revision key with optional metadata TTL."""
    try:
        if _FAKE_REVISION_TTL_SECONDS <= 0:
            path = FAKE_CSV_FILES[dataset]
            stat = path.stat()
            return (
                path,
                str(path.resolve()),
                stat.st_mtime_ns,
                stat.st_size,
            )

        ttl_bucket = int(
            time.monotonic()
            // _FAKE_REVISION_TTL_SECONDS
        )
        return _cached_fake_csv_revision(
            dataset,
            ttl_bucket,
        )

    except OSError as exc:
        path = FAKE_CSV_FILES[dataset]
        raise FakeCsvConnectorError(
            (
                "Fake connector file is missing: "
                f"{path}. Restore it or replace the "
                f"{dataset!r} loader with a real function."
            )
        ) from exc

```

For local disk, leave:

```
export CUBE_FAKE_REVISION_TTL_SECONDS=0
```

For slow network metadata, test:

```
export CUBE_FAKE_REVISION_TTL_SECONDS=1
```

34. Keep one Gunicorn process and make request threads configurable

File: `s04_server.py`

Replace the complete file with:

```
"""Gunicorn settings for one authoritative process-local Cube snapshot."""

import os

workers = 1
worker_class = "gthread"
threads = int(os.getenv("GUNICORN_THREADS", "4"))

timeout = int(
    os.getenv(
        "GUNICORN_TIMEOUT_SECONDS",
        "300",
    )
)
graceful_timeout = int(
    os.getenv(
        "GUNICORN_GRACEFUL_TIMEOUT_SECONDS",
        "30",
    )
)
keepalive = int(
    os.getenv(
        "GUNICORN_KEEPALIVE_SECONDS",
        "5",
    )
)

# Do not preload or automatically recycle this process while the financial
# snapshot is process-local. Either action would trigger another cold start.
preload_app = False
max_requests = 0
```

This preserves the current four-thread default while allowing a controlled eight-thread benchmark. HTTP progress polling, Dash callbacks, and navigation continue to share one authoritative refresh manager.

Do not use multiple Gunicorn worker processes until the following are moved into shared storage:

- snapshot and revision;
- startup ownership;
- refresh lock and progress;
- prepared dashboard cache;
- Quick Search catalogue;
- Stock cache and request intent;
- last-good snapshot.

A future multi-process architecture should use Redis or another shared service and distribute revision-tagged immutable data. That is a separate project, not a one-line Gunicorn change.

35. Add faster Dash JSON serialization

File: requirements.txt

Add:

```
orjson==3.11.9
```

Do not change Dash, pandas, NumPy, or Plotly versions in the same patch.

Dash uses `orjson` automatically when it is installed.

36. Add final-stage timing logs

These logs prove whether the remaining cold-start delay is connector I/O, release aggregation, search-catalogue construction, or browser layout construction.

File: core/s02_pipeline.py

The module already imports `time` .

In `RiskRefreshManager.refresh()` , find the `_progress_step()` call whose function name is `_commit_full_snapshot` and whose stage is final

Immediately before that call, add:

```
final_started = time.perf_counter()
```

Immediately before `_release_pl_views(...)` , add:

```
release_started = time.perf_counter()
```

Immediately after `_release_pl_views(...)` , add:

```
release_seconds = time.perf_counter() - release_started
LOGGER.info(
    "Cube release views completed in %.3fs; combined_pl=%s dashboard=%s "
    "unmapped=%s",
    release_seconds,
    len(enriched),
    len(dashboard),
    len(unmapped),
)
```

Immediately before `_build_snapshot_search_catalog(...)` , add:

```
search_started = time.perf_counter()
```

Immediately after it, add:

```
search_seconds = time.perf_counter() - search_started
LOGGER.info(
    "Cube search catalogue completed in %.3fs; dashboard_rows=%s",
    search_seconds,
    len(dashboard),
)
```

Immediately after `_commit_full_snapshot(...)` , add:

```
LOGGER.info(
    "Cube final stage completed in %.3fs; revision=%s",
    time.perf_counter() - final_started,
    revision,
```

```
)
```

File: `ui/s07_events.py`

Inside `materialize_initial_dashboard()` , before `read_dashboard()` , add:

```
ui_started = monotonic()
```

After `build_layout(...)` , add:

```
app.logger.info(
    "Cube initial dashboard UI built in %.3fs; revision=%s rows=%s",
    monotonic() - ui_started,
    dashboard_read.revision,
    len(prepared),
)
```

`monotonic` is already imported in this module.

37. Reduce the startup watchdog from forty minutes

The watchdog does not kill a connector; it only marks the startup as stalled. After adding connector-level timeouts and bounded task batches, forty minutes is too long for useful diagnosis.

File: `ui/s09_factory.py`

Replace both default values of `2400` in startup-timeout parsing with `600` :

```
raw_timeout = os.getenv(
    "CUBE_STARTUP_TIMEOUT_SECONDS",
    "600",
)
```

and:

```
startup_timeout = 600.0
```

Use a larger value through the environment only if measured successful cold starts legitimately exceed ten minutes.

38. Optional cleanup of recovered AI/comment blocks

Do this only after the functional and cold-start patches pass.

Delete the enormous comment-only connector implementations from:

- `adapters/s01_common.py` .
- `adapters/s02_ir.py` .
- `adapters/s03_fx.py` .
- `adapters/s04_credit.py` .
- `feeds/s01_sources.py` .

Delete recovered comment-only alternatives from `core/s02_pipeline.py` , including:

- MRX/MMM field alternatives.
- unused formula-name alternatives.

- recovered ProductSpec alternatives.
- recovered Age arithmetic.

Keep the active adapter contracts and the new `run_async` implementation.

If you perform this cleanup, also update:

- `README.md` , which currently claims recovered bodies are kept inline;
- `tests/s20_connector_provenance.py` , which explicitly requires those blocks;
- `tests/s21_pipeline_provenance.py` , which explicitly requires recovered pipeline markers;
- `tests/s13_publish.py` , which checks selected marker text in the staged runtime.

Do not delete these useful structures:

- `HierarchyAggregationIndex` .
- `_MarketQuoteIndex` .
- revision-specific prepared dashboard caching.
- `StartupCoordinator`.
- atomic snapshot publication.
- last-good-snapshot behavior.
- Stock stale-request protection.
- lazy detailed-row loading.

39. Update the tests

Run test changes in the same commits as their behaviour changes.

39.1 Portfolio registry and release

File: `tests/s19_risk_filters.py`

Add:

```
assert "portfolio" not in {
    field.key
    for field in FILTER_DIMENSION_FIELDS
}

assert "portfolio" not in {
    field.key
    for field in VIEW_DIMENSION_FIELDS
}
```

Replace Portfolio filter examples with Activity or Category.

Delete tests for `filter_unmapped_portfolios` .

39.2 Add a dashboard collapse integration test

Use two mapped rows that differ only by Portfolio:

```
def test_dashboard_collapses_portfolio_after_mapping():
    dashboard = to_dashboard_frame(
        mapped_two_portfolio_frame()
    )
```

```

assert "Portfolio" not in dashboard.columns
assert len(dashboard) == 1
assert dashboard["Risk"].iloc[0] == 30.0
assert dashboard["dRisk"].iloc[0] == 3.0
assert dashboard["PL"].iloc[0] == 10.0
assert dashboard["Open"].iloc[0] == 3.0
assert dashboard["Current"].iloc[0] == 4.0
assert dashboard["Move"].iloc[0] == 1.0

```

Do not compare all `combined_pl` totals to dashboard totals when unmapped Portfolio rows exist. Compare only mapped input rows.

39.3 Saved views

File: `tests/s23_saved_views.py`

Expected keys become:

```

(
    "activity",
    "signoffgroup",
    "category",
    "subcategory",
)

```

Remove Portfolio from fixtures and expected JSON.

Change the CSS assertion from five columns to four.

39.4 Partial Credit

File: `tests/s06_ui.py`

Add:

```

def test_partial_credit_measure_remains_available():
    frame = credit_frame().copy()
    frame.loc[0, "risk sp01"] = np.nan

    assert credit_measure_available(
        frame,
        "SP01",
    )

    values = credit_measure_values(
        frame,
        "risk",
        "SP01",
    )

    assert pd.isna(values.iloc[0])
    assert values.iloc[1:].notna().all()

```

Add an all-missing test that expects the measure to be unavailable.

39.5 Market duplicate and tenor tests

File: `tests/s04_market.py`

Replace tests expecting failure for:

- duplicate market keys
- conflicting tenor ranks
- Open/Current rank disagreement

Add tests asserting:

```
def test_duplicate_open_quotes_are_averaged():
    result = get_product_market_open(
        spec,
        market_date,
        duplicate_open_frame,
    )

    assert result["Open"].iloc[0] == pytest.approx(
        duplicate_open_frame["Open"].mean()
    )

def test_duplicate_current_quotes_are_averaged():
    result = get_product_market_status(
        spec,
        market_date,
        duplicate_current_frame,
        market_status="Live",
    )

    assert result["Current"].iloc[0] == pytest.approx(
        duplicate_current_frame["Current"].mean()
    )

def test_misaligned_orders_are_normalized():
    result = get_product_market(
        spec,
        market_date,
        open_frame_with_conflicts,
        current_frame_with_conflicts,
        market_status="Live",
    )

    orders = result[
        "Tenor Swap Order"
    ].drop_duplicates().sort_values().tolist()

    assert orders == list(range(len(orders)))
```

39.6 Reported Underlying market aggregate

File: tests/s06_ui.py

Create several raw Underlyings under one Reported Underlying and assert Open, Current and Move appear at the Reported Underlying row.

39.7 Cold startup shell

File: tests/s12_startup.py

Cold shell:

```
assert progress.hidden is False
assert (
    progress.to_plotly_json()["props"]
    ["data-initial-load"]
    == "true"
)
assert bootstrap_interval.disabled is False
```

Warm shell:

```
assert progress.hidden is True
```

Change the JavaScript assertion to:

```
assert "hasNewError ? 5000 : 1500" in source
assert "Date.now() + 60000" in source
```

Add a test that `serve_layout()` calls `schedule_start()` once for a cold manager and not for a warm manager.

39.8 Targeted dashboard read

Add a test that `read_dashboard()` returns one dashboard frame and compact control view without exposing the large P&L, MarketBook or unmapped frames.

Add:

```
dashboard_read = manager.read_dashboard()

assert dashboard_read.revision == manager.health.revision
assert isinstance(dashboard_read.frame, pd.DataFrame)
assert dashboard_read.control.revision == dashboard_read.revision
assert not hasattr(dashboard_read, "combined_pl")
assert not hasattr(dashboard_read, "market_frame")
assert not hasattr(dashboard_read, "unmapped_frame")
```

Mutate the returned `frame` and `control.risk_status` in the test, then call `read_dashboard()` again and assert the committed manager values were not changed.

39.9 Startup no-copy return

Update all test fake-manager `refresh()` signatures to accept:

```
copy_result: bool = True,
```

Add a test that `StartupCoordinator` calls revision 1 with `copy_result=False` .

39.10 Connector concurrency

Add a bounded-concurrency test:

```
from threading import Lock
from time import sleep

def test_market_connector_uses_bounded_workers():
    lock = Lock()
    active = 0
    peak = 0

    def market_open(
        market_date,
        underlying,
        *,
        market_status,
    ):
        nonlocal active, peak

        with lock:
            active += 1
            peak = max(peak, active)

        try:
            sleep(0.05)
            return open_frame_for(underlying)
        finally:
            with lock:
                active -= 1

    manager = build_manager(
        market_open_loader=market_open,
        market_workers=3,
    )

    manager.refresh(
        expected_revision=0,
        copy_result=False,
    )

    assert 1 < peak <= 3
```

Add the same test with `market_workers=1` and assert `peak == 1` .

39.11 Stock

File: tests/s17_stock.py

Remove hierarchy-path, promotion-threshold and temporary-group tests.

Add tests asserting:

- duplicate raw identities are summed
- Portfolio is still used during mapping
- mapped public output has no Portfolio column
- Activity aggregate equals the sum of detailed collapsed rows
- prior/current/change values are correct
- the page contains stock-aggregate-dimension
- the page contains stock-aggregate-grid
- the page has no stock-hierarchy-open-paths
- the page has no stock-promotion-threshold

39.12 Fake fixture mode

In test setup, set:

```
monkeypatch.setenv(
    "CUBE_RISK_CONNECTOR_WORKERS",
    "1",
)
monkeypatch.setenv(
    "CUBE_MARKET_CONNECTOR_WORKERS",
    "1",
)
monkeypatch.setenv(
    "CUBE_STOCK_CONNECTOR_WORKERS",
    "1",
)
```

This keeps fixture tests deterministic. Concurrency has its own explicit tests.

40. Run checks in this order

40.1 Compile

```
python -m compileall \
    adapters \
    core \
    feeds \
    pages \
    ui
```

40.2 Focused tests

```
python -m pytest \
    tests/s04_market.py \
    tests/s05_pl.py \
    tests/s06_ui.py \
    tests/s07_integration.py \
    tests/s12_startup.py \
    tests/s17_stock.py \
    tests/s19_risk_filters.py \
    tests/s23_saved_views.py \
    -q
```

40.3 Complete suite

```
python -m pytest -q
```

40.4 Stage the deployment bundle

```
python s03_publish.py \  
--keep-bundle /tmp/rebirth-bundle
```

Inspect:

```
find /tmp/rebirth-bundle/runtime \  
-maxdepth 2 \  
-type f \  
| sort
```

Then publish using the normal command.

41. Recommended environment settings

41.1 Fake/local fixture mode

```
export CUBE_RISK_CONNECTOR_WORKERS=1  
export CUBE_MARKET_CONNECTOR_WORKERS=1  
export CUBE_FAKE_REVISION_TTL_SECONDS=0  
export RISK_PRODUCT_DELAY_SECONDS=0  
export GUNICORN_THREADS=4  
export CUBE_STARTUP_TIMEOUT_SECONDS=600
```

41.2 Initial real-source mode

Use only after confirming private client thread-safety and rate limits:

```
export CUBE_RISK_CONNECTOR_WORKERS=2  
export CUBE_MARKET_CONNECTOR_WORKERS=4  
export CUBE_STOCK_CONNECTOR_WORKERS=3  
export CUBE_CONNECTOR_BATCH_TIMEOUT_SECONDS=300  
export CUBE_ASYNC_CONNECTOR_TIMEOUT_SECONDS=120  
export CUBE_PORTFOLIO_BATCH_TIMEOUT_SECONDS=180  
export CUBE_STOCK_BATCH_TIMEOUT_SECONDS=180  
export CUBE_USE_REAL_PORTFOLIO_CONFIG=1  
export RISK_PRODUCT_DELAY_SECONDS=0  
export GUNICORN_THREADS=4  
export GUNICORN_TIMEOUT_SECONDS=300  
export CUBE_STARTUP_TIMEOUT_SECONDS=600
```

After the application is stable, benchmark each change separately:

```
export CUBE_RISK_CONNECTOR_WORKERS=4  
export CUBE_MARKET_CONNECTOR_WORKERS=8
```

Then restore the best connector values and benchmark:

```
export GUNICORN_THREADS=8
```

Do not change Gunicorn process count.

42. Production acceptance checklist

Cold start

- `/healthz` responds before the first source call completes.
- `/progressz` changes at least once every normal source-duration interval.
- `startup_phase` changes from `idle` to `running` without requiring a manual refresh.
- a source timeout produces a visible failed startup instead of an indefinite wait.
- revision changes from 0 to 1.
- the hero remains visible until the page has adopted revision 1.
- the completion state remains visible for 1.5 seconds.
- the browser does not enter a reload loop.

Cardinality

- `combined_pl` contains Portfolio.
- `dashboard_frame` does not contain Portfolio.
- the log prints `Dashboard rows after removing Portfolio: before -> after` .
- mapped Risk/dRisk/P&L totals are preserved.
- Quick Search does not require Portfolio.
- Risk, P&L and Stock filters do not contain Portfolio.

Credit and market

- one missing SP01/PSP01/PM01 leaf does not hide the measure.
- all-missing measures remain unavailable.
- duplicate quotes are averaged.
- Move equals Current minus Open after aggregation.
- conflicting tenor ranks do not abort refresh.
- final tenor ranks are unique and consecutive.
- Reported Underlying displays aggregated market values.

Concurrency

- real source clients have native timeouts.
- market concurrency never exceeds `CUBE_MARKET_CONNECTOR_WORKERS` .
- fake tests use one connector worker.
- upstream services are not overloaded.
- one connector failure cancels the transaction and no partial snapshot is committed.

Stock

- duplicate raw Stock rows are summed.

- Portfolio is removed after mapping.
- Aggregate Stock has the same View-by vocabulary as P&L.
- detailed rows remain lazy.
- promotion and custom hierarchy state are gone.
- the Stock cache contains at most two comparisons.

Server

- Gunicorn uses one process.
- Gunicorn uses `gthread` .
- thread count is configurable.
- `preload_app` is false.
- no automatic worker recycling recreates unnecessary cold starts.

43. Expected result

After completing the guide:

- a cold worker becomes HTTP-reachable before connectors finish
- startup begins from the persistent shell, JavaScript endpoint, page-local interval, or delayed server fallback without duplicate writers
- a hung private source fails on a bounded timeout instead of freezing forever
- real Risk and market I/O can overlap through bounded connector threads
- the fake fixture remains simple, cached and deterministic
- the browser no longer waits for full copies of unrelated frames
- the Risk and P&L pages no longer build initial tables twice
- Portfolio disappears from the browser-facing analytical cube and Quick Search
- partial Credit measures remain visible
- duplicate market quotes are averaged
- tenor order conflicts are normalized
- Reported Underlying displays market aggregates
- Stock becomes a simple P&L-style aggregate page
- one Gunicorn process remains the authoritative owner of in-memory state
- Dash uses faster JSON serialization

44. Primary references

Repository files reviewed at commit `bd8ae49b5c743471afcc831f7e6bb50f56e35eb3` :

- `s01_app.py`
- `s02_config.py`
- `s03_publish.py`

- `s04_server.py`
- `feeds/s01_sources.py`
- `core/s01_schema.py`
- `core/s02_pipeline.py`
- `core/s03_search.py`
- `core/s07_stock.py`
- `ui/s01_contracts.py`
- `ui/s02_constants.py`
- `ui/s03_aggregate.py`
- `ui/s04_components.py`
- `ui/s06_plview.py`
- `ui/s07_events.py`
- `ui/s08_plevents.py`
- `ui/s09_factory.py`
- `ui/s10_stock.py`
- `assets/s02_app.js`

Official documentation used for the deployment recommendations:

- Python `concurrent.futures` : <https://docs.python.org/3/library/concurrent.futures.html>
- Gunicorn settings: <https://docs.gunicorn.org/en/stable/settings.html>
- Gunicorn design: <https://docs.gunicorn.org/en/stable/design.html>
- Dash performance: <https://dash.plotly.com/performance>
- Dash data sharing and multi-process memory: <https://dash.plotly.com/sharing-data-between-callbacks>
- Plotly Cloud runtime logs: <https://dash.plotly.com/plotly-cloud/logs>